```
# ==== run.py ====
# -*- coding: utf-8 -*-
"""系统启动入口。
运行：
    python run.py
然后在浏览器打开 http://127.0.0.1:5000
"""
from app import create_app
from app.models import db
import sys
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")  # 控制台中文正常显示
app = create_app()
if __name__ == "__main__":
    with app.app_context():
        db.create_all()  # 首次运行时自动建表
    print("=" * 58)
    print("  历史遗留废弃矿山自然恢复适宜性评价系统 V1.0")
    print("=" * 58)
    print("  访问地址：http://127.0.0.1:5000")
    print("  演示账号：admin  密码：admin123")
    print("  停止运行：按 Ctrl+C")
    print("=" * 58)
    app.run(host="127.0.0.1", port=5000, debug=True)
# ==== app/__init__.py ====
# -*- coding: utf-8 -*-
"""Flask 应用工厂。"""
from flask import Flask
from flask_login import LoginManager
from app.config import Config
from app.models import db, User
login_manager = LoginManager()
login_manager.login_view = "auth.login"
login_manager.login_message = "请先登录"
@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)
    login_manager.init_app(app)
    # 注册视图（各模块路由）
    from app.views import auth, project, unit, standard,
    constraint, \
        ahp, indicator, evaluate, report
    app.register_blueprint(auth.bp)
    app.register_blueprint(project.bp)
    app.register_blueprint(unit.bp)
    app.register_blueprint(standard.bp)
```

```
    app.register_blueprint(constraint.bp)
    app.register_blueprint(ahp.bp)
    app.register_blueprint(indicator.bp)
    app.register_blueprint(evaluate.bp)
    app.register_blueprint(report.bp)
    #  全局登录保护：除登录页与静态资源外，均需登录
    from flask import request, redirect, url_for
    from flask_login import current_user, login_user
    PUBLIC_ENDPOINTS = {"auth.login", "static"}
    @app.before_request
    def require_login():
        #  截图模式（仅用于生成说明书配图）：自动以管理员身份登录
        if app.config.get("SCREENSHOT_MODE"):
            if (request.endpoint not in ("auth.login",
"static")
                    and not current_user.is_authenticated):
                admin = User.query.filter_by(role="admin").f
irst()
                if admin:
                    login_user(admin)
            return None
        if request.endpoint in PUBLIC_ENDPOINTS or
request.endpoint is None:
            return None
        if not current_user.is_authenticated:
            return redirect(url_for("auth.login"))
        return None
    #  侧边栏导航项：集中定义，模板只负责渲染；active 用于高亮当前页
    @app.context_processor
    def inject_nav():
        path = request.path or "/"
        items = [
            ("首页", "/", path == "/"),
            ("项目与档案", "/project/", path.startswith("/project
")),
            ("限制性判定", "/constraint/",
path.startswith("/constraint")),
            ("标准知识库", "/standard/", path.startswith("/standa
rd")),
            ("AHP 权重", "/ahp/", path.startswith("/ahp")),
            ("指标数据", "/indicator/", path.startswith("/indica
tor")),
            ("适宜性评价", "/evaluate/", path.startswith("/evalua
te")),
            ("报告输出", "/report/", path.startswith("/report"))
,
            ("用户管理", "/users", path.startswith("/users")),
        ]
        return {"nav_items": items}
    return app
```

```
# ==== app/config.py ====
# -*- coding: utf-8 -*-
"""应用配置。"""
import os
from datetime import timedelta
BASE_DIR = os.path.abspath(os.path.dirname(__file__))
class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY",
    "mine-restoration-dev-key")
    # 单机部署，SQLite 数据库文件放在应用目录下
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE_DIR, "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    # 禁止跨站请求伪造（CSRF），登录与表单提交依赖此项
    WTF_CSRF_ENABLED = True
    # 「记住我」：登录时勾选则下发持久 cookie，关闭浏览器后仍可自动登录
    REMEMBER_COOKIE_DURATION = timedelta(days=30)
    REMEMBER_COOKIE_HTTPONLY = True
    REMEMBER_COOKIE_SAMESITE = "Lax"
# ==== app/models.py ====
# -*- coding: utf-8 -*-
"""数据模型：系统全部数据表。
本文件由甲在阶段 0 一次性写完，作为两人独立开发的地基。
约定：两人只读此文件；确需改表时，由乙提需求、甲统一修改。
"""
import json
from datetime import datetime
from flask_login import UserMixin
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
class User(UserMixin, db.Model):
    """用户（M1 用户与权限管理）。"""
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True,
    nullable=False, index=True)
    password_hash = db.Column(db.String(256),
    nullable=False)
    # 角色：管理员 admin / 评价员 evaluator / 查看者 viewer
    role = db.Column(db.String(20), nullable=False,
    default="viewer")
    active = db.Column(db.Boolean, nullable=False,
    default=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    def to_dict(self):
        return {"id": self.id, "username": self.username,
                "role": self.role, "active": self.active}
class OperationLog(db.Model):
    """操作日志（M1）。"""
```

```
    __tablename__ = "operation_logs"
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"
    ), nullable=True)
    action = db.Column(db.String(128), nullable=False)
    # 操作类型
    target_type = db.Column(db.String(64), nullable=True)
    # 对象类型
    target_id = db.Column(db.Integer, nullable=True)
    # 对象主键
    detail = db.Column(db.Text, nullable=True)
    # 操作详情
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow, index=True)
class Project(db.Model):
    """评价项目（M2 项目与矿山档案）。"""
    __tablename__ = "projects"
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(128), nullable=False)
    code = db.Column(db.String(64), nullable=True)
    # 项目编号
    description = db.Column(db.Text, nullable=True)
    status = db.Column(db.String(20), default="进行中")
    # 进行中 / 已完成 / 已归档
    created_by = db.Column(db.Integer,
    db.ForeignKey("users.id"), nullable=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    patches = db.relationship("MinePatch",
    backref="project", lazy="dynamic",
                              cascade="all, delete-orphan")
class MinePatch(db.Model):
    """矿山图斑（M2）。"""
    __tablename__ = "mine_patches"
    id = db.Column(db.Integer, primary_key=True)
    project_id = db.Column(db.Integer,
    db.ForeignKey("projects.id"), nullable=False)
    name = db.Column(db.String(128), nullable=False)
    admin_region = db.Column(db.String(128), nullable=True)
    # 所在行政区
    center_x = db.Column(db.Float, nullable=True)
    # 中心经度 / 横坐标
    center_y = db.Column(db.Float, nullable=True)
    # 中心纬度 / 纵坐标
    area_ha = db.Column(db.Float, nullable=True)
    # 图斑面积（公顷）
    mineral_type = db.Column(db.String(64), nullable=True)
    # 开采矿种
    mining_method = db.Column(db.String(64), nullable=True)
    # 开采方式
```

```
    stop_years = db.Column(db.Float, nullable=True)
    # 停产年限
    responsible_entity_missing = db.Column(db.Boolean,
    default=True)  # 责任主体是否缺失
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    units = db.relationship("MicroUnit", backref="patch",
    lazy="dynamic",
                            cascade="all, delete-orphan")
class ReferenceEcosystem(db.Model):
    """参照生态系统（M2）：标准化基准值，与图斑绑定。"""
    __tablename__ = "reference_ecosystems"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    coverage_benchmark = db.Column(db.Float, default=75.0)
    # 植被覆盖度基准 %
    shannon_benchmark = db.Column(db.Float, default=0.7)
    # Shannon-Wiener 基准
    dominant_species = db.Column(db.String(256),
    nullable=True)  # 建群种
    soil_benchmark = db.Column(db.String(256),
    nullable=True)    # 土壤基准
class Standard(db.Model):
    """地方标准 / 政策文件（M3 政策与标准知识库）。"""
    __tablename__ = "standards"
    id = db.Column(db.Integer, primary_key=True)
    province = db.Column(db.String(32), nullable=False)
    # 省份
    name = db.Column(db.String(256), nullable=False)
    # 名称
    doc_number = db.Column(db.String(128), nullable=True)
    # 标准编号 / 文号
    publish_date = db.Column(db.String(32), nullable=True)
    implement_date = db.Column(db.String(32), nullable=True)
    # 类别：地方标准 / 政策文件
    category = db.Column(db.String(32), nullable=False,
    default="地方标准")
    summary = db.Column(db.Text, nullable=True)
    # 特点概述
    rules = db.relationship("StandardRule",
    backref="standard",
                            cascade="all, delete-orphan")
class StandardRule(db.Model):
    """结构化规则（M3）：将标准条文拆解为可执行规则。"""
    __tablename__ = "standard_rules"
    id = db.Column(db.Integer, primary_key=True)
    standard_id = db.Column(db.Integer,
    db.ForeignKey("standards.id"), nullable=False)
    item = db.Column(db.String(128), nullable=False)
```

```
    # 判定项
    operator = db.Column(db.String(8), nullable=False)
    # 比较符：> >= < <= ==
    threshold = db.Column(db.String(64), nullable=True)
    # 阈值
    condition_type = db.Column(db.String(32),
    default="必要条件")  # 必要条件 / 充分条件
    region = db.Column(db.String(64), nullable=True)
    # 适用分区（宁夏分区差异化）
    note = db.Column(db.Text, nullable=True)
class ConstraintCheck(db.Model):
    """限制性判定记录（M4）。"""
    __tablename__ = "constraint_checks"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    policy_pass = db.Column(db.Boolean, nullable=True)
    # 政策约束
    location_pass = db.Column(db.Boolean, nullable=True)
    # 区位约束
    pollution_pass = db.Column(db.Boolean, nullable=True)
    # 土壤污染
    geo_pass = db.Column(db.Boolean, nullable=True)
    # 地质灾害
    farmland_pass = db.Column(db.Boolean, nullable=True)
    # 耕地属性
    passed = db.Column(db.Boolean, nullable=True)
    # 总体是否通过
    reason = db.Column(db.Text, nullable=True)
    # 不通过原因
    checked_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class MicroUnit(db.Model):
    """微地质单元（M5）：岩质边坡 / 碎石边坡 / 平台。"""
    __tablename__ = "micro_units"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    unit_type = db.Column(db.String(32), nullable=False)
    # 单元类型
    area = db.Column(db.Float, nullable=True)
    # 面积
    ratio = db.Column(db.Float, nullable=True)
    # 面积占比（自动计算）
    description = db.Column(db.Text, nullable=True)
    # 现场描述
class Indicator(db.Model):
    """指标定义（M6）：指标树，含角色与启用单元类型。"""
    __tablename__ = "indicators"
    id = db.Column(db.Integer, primary_key=True)
```

```
    code = db.Column(db.String(32), unique=True,
    nullable=False)  # 指标编码
    name = db.Column(db.String(64), nullable=False)
    # 指标名称
    level = db.Column(db.Integer, nullable=False)
    # 1/2/3 级
    parent_code = db.Column(db.String(32), nullable=True)
    # 父指标编码
    # 角色：土壤污染指标同时作为限制性指标与评价指标
    is_constraint = db.Column(db.Boolean, default=False)
    is_evaluative = db.Column(db.Boolean, default=True)
    unit = db.Column(db.String(32), nullable=True)
    # 单位
    ideal_value = db.Column(db.String(64), nullable=True)
    # 理想值（可配置）
    ideal_type = db.Column(db.String(32), nullable=True)
    # 标准化类型：benefit/cost/mid/discrete
class WeightScheme(db.Model):
    """权重方案（M6）：按微地质单元类型分别保存。"""
    __tablename__ = "weight_schemes"
    id = db.Column(db.Integer, primary_key=True)
    unit_type = db.Column(db.String(32), nullable=False)
    # 所属单元类型
    name = db.Column(db.String(64), nullable=True)
    # 方案名称
    level = db.Column(db.Integer, nullable=False,
    default=1)        # 1=一级指标 2=二级指标
    is_default = db.Column(db.Boolean, default=False)
    # 是否默认（论文表 3）
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class WeightItem(db.Model):
    """权重明细（M6）：方案内的各项权重。"""
    __tablename__ = "weight_items"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=False)
    indicator_code = db.Column(db.String(32),
    nullable=False)       # 指标编码
    weight = db.Column(db.Float, nullable=False)
    # 权重值
class ExpertMatrix(db.Model):
    """专家判断矩阵（M6）：多专家打分的原始矩阵。"""
    __tablename__ = "expert_matrices"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=False)
    expert_name = db.Column(db.String(64), nullable=True)
    # 专家姓名
    level = db.Column(db.Integer, nullable=False,
```

```
    default=1)          #  矩阵对应指标层级                                    351
    matrix_json = db.Column(db.Text, nullable=False)                          352
    #  判断矩阵（JSON 二维数组）                                               353
    created_at = db.Column(db.DateTime,                                       354
    default=datetime.utcnow)                                                  355
class IndicatorData(db.Model):                                                356
    """指标实测数据（M7）：按微地质单元组织。"""                                 357
    __tablename__ = "indicator_data"                                          358
    id = db.Column(db.Integer, primary_key=True)                              359
    unit_id = db.Column(db.Integer, db.ForeignKey("micro_uni                  360
    ts.id"), nullable=False, unique=True)                                     361
    soil_ratio = db.Column(db.Float, nullable=True)                           362
    #  土石比                                                                 363
    thickness = db.Column(db.Float, nullable=True)                            364
    #  土层厚度  cm                                                           365
    ph = db.Column(db.Float, nullable=True)                                   366
    organic_matter = db.Column(db.Float, nullable=True)                       367
    #  有机质  g/kg                                                           368
    #  重金属含量，JSON 字符串：{"Hg": 0.05, ...}，未检测项缺失或为 null          369
    heavy_metals = db.Column(db.Text, nullable=True)                          370
    coverage = db.Column(db.Float, nullable=True)                             371
    #  植被覆盖度 %                                                           372
    shannon = db.Column(db.Float, nullable=True)                              373
    #  Shannon-Wiener 多样性指数                                              374
    slope = db.Column(db.Float, nullable=True)                                375
    #  坡度 °                                                                 376
    aspect = db.Column(db.String(16), nullable=True)                          377
    #  坡向：阳坡/阴阳坡/阴坡                                                 378
    def heavy_metals_dict(self):                                              379
        return json.loads(self.heavy_metals) if                               380
    self.heavy_metals else {}                                                 381
class EvaluationResult(db.Model):                                             382
    """评价结果（M8）。"""                                                     383
    __tablename__ = "evaluation_results"                                      384
    id = db.Column(db.Integer, primary_key=True)                              385
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat                  386
    ches.id"), nullable=False)                                                387
    final_score = db.Column(db.Float, nullable=True)                          388
    #  最终评价因子 F                                                         389
    grade = db.Column(db.String(32), nullable=True)                           390
    #  适宜性等级                                                             391
    measures = db.Column(db.Text, nullable=True)                              392
    #  推荐措施                                                               393
    unit_scores = db.Column(db.Text, nullable=True)                           394
    #  各单元得分 JSON                                                        395
    created_at = db.Column(db.DateTime,                                       396
    default=datetime.utcnow)                                                  397
class ScoreDetail(db.Model):                                                  398
    """得分明细（M8）：每一层指标的权重、标准化值、加权贡献，供追溯与复核。"""    399
    __tablename__ = "score_details"                                           400
```

```
    id = db.Column(db.Integer, primary_key=True)
    result_id = db.Column(db.Integer,
    db.ForeignKey("evaluation_results.id"), nullable=False)
    unit_id = db.Column(db.Integer, nullable=True)
    l1_name = db.Column(db.String(64), nullable=True)
    # 一级指标
    l2_name = db.Column(db.String(64), nullable=True)
    # 二级指标
    raw_value = db.Column(db.String(64), nullable=True)
    # 实测值
    std_value = db.Column(db.Float, nullable=True)
    # 标准化值
    weight = db.Column(db.Float, nullable=True)
    # 权重
    contribution = db.Column(db.Float, nullable=True)
    # 加权贡献
# ==== app/forms.py ====
# -*- coding: utf-8 -*-
"""表单定义（共享，由甲统一维护）。
各模块的 Flask-WTF 表单集中于此。
"""
from flask_wtf import FlaskForm
from wtforms import SelectField, SubmitField, StringField,
    PasswordField, BooleanField
from wtforms.validators import DataRequired, Length
from app.utils.constants import UNIT_TYPES, ASPECT_TYPES
class AHPSelectForm(FlaskForm):
    """M6：选择微地质单元类型与指标层级，以确定判断矩阵规模。"""
    unit_type = SelectField("微地质单元类型",
    validators=[DataRequired()])
    level = SelectField("指标层级",
                        choices=[("1", "一级指标"), ("2",
    "二级指标")],
                        validators=[DataRequired()])
    l1 = SelectField("一级指标", validators=[],
    validate_choice=False)  # 动态填充，层级=2 时使用
    submit = SubmitField("生成判断矩阵")
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.unit_type.choices = [(t, t) for t in
    UNIT_TYPES]
class EmptyForm(FlaskForm):
    """仅用于承载 CSRF 令牌的空表单（矩阵录入等自定义表单使用）。"""
    pass
class LoginForm(FlaskForm):
    """M1：登录表单（占位，由乙完善）。"""
    username = StringField("用户名", validators=[DataRequired()
    , Length(max=64)])
    password = PasswordField("密码", validators=[DataRequired(
    )])
```

```
    remember = BooleanField("记住我")
    submit = SubmitField("登录")
# ==== app/utils/constants.py ====
# -*- coding: utf-8 -*-
"""评价指标常量表。
全部数值取自论文，作为系统计算的唯一基准：
- 论文 4.2.2 节：指标标准化理想极值与取值依据
- 论文 4.2.3 节 / 表 4：适宜性等级划分
- 论文表 3：按微地质单元分别确定的一级、二级指标权重
- 论文 4.2.1 节：平均随机一致性指标 RI
"""
# ---------------------------------------------------------------------
    ----------------
# 一、微地质单元类型
# ---------------------------------------------------------------------
    ----------------
# 论文 4.1 节："按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡
# 三种地质微单元类型"
UNIT_TYPES = ("岩质边坡", "碎石边坡", "平台")
# 每个单元类型启用的指标类别（论文表 3 隐含：岩质边坡无土壤指标，平台无地形指标）
# 供数据录入界面与评价引擎判断"该单元是否需要某类指标"
UNIT_ENABLED_INDICATORS = {
    "岩质边坡": ("植被指标", "地形指标"),                # 不设土壤指标
    "碎石边坡": ("土壤指标", "植被指标", "地形指标"),
    "平台": ("土壤指标", "植被指标"),                    # 不设地形指标
}
# ---------------------------------------------------------------------
    ----------------
# 二、指标标准化理想极值（论文 4.2.2 节）
# ---------------------------------------------------------------------
    ----------------
IDEAL_SOIL_RATIO = 0.8          # 土石比理想值
IDEAL_THICKNESS = 20.0          # 土层厚度理想值 （cm）
IDEAL_PH_RANGE = (5.0, 9.0)     # pH 理想区间（适中型指标）
IDEAL_ORGANIC = 30.0            # 有机质理想值
    （g/kg），全国第二次土壤普查养分等级划分标准二等最低值
IDEAL_COVERAGE = 75.0           # 植被覆盖度参照生态系统 （%）
IDEAL_SHANNON = 0.7             # Shannon-Wiener 多样性指数参照生态系统
# 重金属极值 （mg/kg），《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值
# 论文原文：镉、汞、铅、铬、砷、镍、铜、锌分别为 2、2、530、400、45、220、600、800
HEAVY_METALS = ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu",
    "Zn")
HEAVY_METAL_LIMIT = {
    "Cd": 2.0, "Hg": 2.0, "Pb": 530.0, "Cr": 400.0,
    "As": 45.0, "Ni": 220.0, "Cu": 600.0, "Zn": 800.0,
}
# 污染指标下 8 种重金属"重要性相当，权重平均分配"（论文 4.2.1 节）
HEAVY_METAL_EQUAL_WEIGHT = 1.0 / 8.0
# 坡度分界，论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号
SLOPE_BREAK = 25.0
```

```
# 坡向离散赋值，论文 4.2.2 节："阳坡、阴阳坡、阴坡分别赋值 0.2、0.6、1"
ASPECT_SCORES = {"阳坡": 0.2, "阴阳坡": 0.6, "阴坡": 1.0}
ASPECT_TYPES = ("阳坡", "阴阳坡", "阴坡")
# pH 适中型标准化的容差边界（区间外线性衰减，避免得分突变）
PH_TOL_LOW = 4.0
PH_TOL_HIGH = 10.0
# ---------------------------------------------------------------------
    ------------------
# 三、适宜性等级划分（论文表 4）
# ---------------------------------------------------------------------
    ------------------
GRADE_LEVELS = (
    {"grade": "不适宜", "min": None, "max": 0.3,
     "meaning": "存在限制性条件或区域生态本底较差，自然恢复影响因子严重制约自然恢复",
     "measure": "生态重建、人工辅助"},
    {"grade": "基本适宜", "min": 0.3, "max": 0.7,
     "meaning": "生态本底良好，核心自然恢复影响因子基本满足自然恢复需求",
     "measure": "自然恢复为主，可辅以封闭场地和少量补种、补植、飞播等"},
    {"grade": "适宜", "min": 0.7, "max": None,
     "meaning": "各自然恢复影响因子均能满足自然恢复需求，自然演替速度快",
     "measure": "自然恢复"},
)
def grade_of(score):
    """按论文表 4 判定适宜性等级。"""
    for g in GRADE_LEVELS:
        lo, hi = g["min"], g["max"]
        if lo is None and score < hi:
            return g
        if hi is None and score >= lo:
            return g
        if lo is not None and hi is not None and lo <=
    score < hi:
            return g
    # score 恰好等于 0.7 时落入"适宜"
    return GRADE_LEVELS[-1]
# ---------------------------------------------------------------------
    ------------------
# 四、平均随机一致性指标 RI（论文 4.2.1 节给出 n=2,3,4,5，其余为常用扩展值）
# ---------------------------------------------------------------------
    ------------------
RI_TABLE = {1: 0.0, 2: 0.0, 3: 0.58, 4: 0.90, 5: 1.12, 6:
    1.24,
            7: 1.32, 8: 1.41, 9: 1.45, 10: 1.49}
# 一致性比例阈值（论文：当 CR<0.1 时，判断矩阵满足一致性要求）
CR_THRESHOLD = 0.1
# ---------------------------------------------------------------------
    ------------------
# 五、默认权重方案（论文表 3）
# ---------------------------------------------------------------------
    ------------------
```

```
#  论文明确："划分岩质边坡、碎石土边坡、平台三类微地质单元开展独立评价，
#  分别计算评价指标权重"。权重由　１０　位专家打分后经　ＡＨＰ　方法确定。
#  下表为论文表　３　所载的归一化权重，作为系统默认方案。
ＤＥＦＡＵＬＴ＿ＷＥＩＧＨＴＳ　＝　｛
　　"岩质边坡"：　｛
　　　　"ｌ１"：　｛"植被指标"：　０．４５，　"地形指标"：　０．５５｝，
　　　　"ｌ２"：　｛
　　　　　　"植被指标"：　｛"植被覆盖度"：　０．６７，　"生物多样性"：　０．３３｝，
　　　　　　"地形指标"：　｛"坡度"：　０．６４，　"坡向"：　０．３６｝，
　　　　｝，
　　｝，
　　"碎石边坡"：　｛
　　　　"ｌ１"：　｛"土壤指标"：　０．４４，　"植被指标"：　０．３１，　"地形指标"：　０．２５｝，
　　　　"ｌ２"：　｛
　　　　　　"土壤指标"：　｛"土石比"：　０．２５，　"土层厚度"：　０．２５，　"ｐＨ"：　０．１７，
　　　　　　　　"有机质"：　０．１７，　"污染指标"：　０．１７｝，
　　　　　　"植被指标"：　｛"植被覆盖度"：　０．６４，　"生物多样性"：　０．３６｝，
　　　　　　"地形指标"：　｛"坡度"：　０．６７，　"坡向"：　０．３３｝，
　　　　｝，
　　｝，
　　"平台"：　｛
　　　　"ｌ１"：　｛"土壤指标"：　０．５４，　"植被指标"：　０．４６｝，
　　　　"ｌ２"：　｛
　　　　　　"土壤指标"：　｛"土石比"：　０．２５，　"土层厚度"：　０．２５，　"ｐＨ"：　０．１７，
　　　　　　　　"有机质"：　０．１７，　"污染指标"：　０．１７｝，
　　　　　　"植被指标"：　｛"植被覆盖度"：　０．６１，　"生物多样性"：　０．３９｝，
　　　　｝，
　　｝，
｝
#　－－－
```

```
    upper : list[list[float]] 或 np.ndarray，上三角元素（不含对角线）。
        例：3 阶时传入 [[a12, a13], [a23]]。
        对角线为 1，下三角取倒数（a_ji = 1/a_ij）。
    """
    upper = np.asarray(upper, dtype=float)
    k = upper.shape[0]
    n = int(round((1 + (1 + 8 * k) ** 0.5) / 2))
    A = np.ones((n, n), dtype=float)
    idx = 0
    for i in range(n):
        for j in range(i + 1, n):
            A[i, j] = upper[idx]
            A[j, i] = 1.0 / upper[idx]
            idx += 1
    return A
def weights_and_consistency(A):
    """由判断矩阵 A 计算权重与一致性指标。
    返回
    ----
    dict：{weights: np.ndarray, lambda_max, ci, cr, n,
    consistent: bool}
    """
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    # 式(2) 行元素乘积
    row_prod = np.prod(A, axis=1)
    # 式(3) 开 n 次方根
    root = row_prod ** (1.0 / n)
    # 式(4) 归一化
    weights = root / root.sum()
    # 式(5) 最大特征值 λmax = (1/n) Σ (A·W)_i / W_i
    aw = A.dot(weights)
    lambda_max = float(np.sum(aw / weights) / n)
    # 式(6) 一致性指标 CI
    ci = (lambda_max - n) / (n - 1) if n > 1 else 0.0
    # 式(7) 一致性比例 CR = CI / RI
    ri = RI_TABLE.get(n, 1.49)
    cr = ci / ri if ri else 0.0
    return {
        "weights": weights,
        "lambda_max": lambda_max,
        "ci": ci,
        "cr": cr,
        "n": n,
        "consistent": cr < CR_THRESHOLD,
    }
def find_most_inconsistent(A):
    """定位判断矩阵中最不一致的一对比较。
    依据：完全一致矩阵应满足 a_ij = w_i / w_j。
    计算每个上三角元素与其理想值的相对偏差，返回偏差最大者。
```

```
    返回
    ----
    dict：{i, j, value, expected, deviation}，偏差最大的 (i, j)
    及对应取值。
    """
    A = np.asarray(A, dtype=float)
    r = weights_and_consistency(A)
    w = r["weights"]
    n = A.shape[0]
    best = None
    for i in range(n):
        for j in range(i + 1, n):
            expected = w[i] / w[j]
            if expected == 0:
                continue
            deviation = abs(A[i, j] - expected) / expected
            if best is None or deviation >
    best["deviation"]:
                best = {"i": i, "j": j, "value": A[i, j],
                        "expected": expected, "deviation":
    deviation}
    return best
def aggregate_matrices(matrices):
    """多专家判断矩阵聚合：几何平均（保持互反性）。
    参数
    ----
    matrices : list[np.ndarray]，多个同阶判断矩阵。
    """
    stack = np.stack([np.asarray(m, dtype=float) for m in
    matrices])
    return np.prod(stack, axis=0) ** (1.0 / len(stack))
def judge_from_weights(weights):
    """由权重向量构造完全一致判断矩阵并检验。
    用途：验证给定权重向量能否作为某个完全一致判断矩阵的主特征向量
    （A_ij = w_i / w_j 时 λmax = n、CR = 0）。
    """
    w = np.asarray(weights, dtype=float)
    n = w.shape[0]
    A = np.outer(w, 1.0 / w)
    r = weights_and_consistency(A)
    r["matrix"] = A
    return r
# ==== app/services/standardization.py ====
# -*- coding: utf-8 -*-
"""指标标准化（M8 的一部分）。
按论文 4.2.2 节式(8) 实现：
    x' = (x - x_min) / (x_max - x_min)
按指标性质分为四类：越大越优 / 越小越优 / 适中 / 离散赋值。
所有理想极值取自 app.utils.constants，可配置。
"""
```

```
from app.utils.constants import (
    IDEAL_SOIL_RATIO, IDEAL_THICKNESS, IDEAL_PH_RANGE,
    IDEAL_ORGANIC,
    IDEAL_COVERAGE, IDEAL_SHANNON, HEAVY_METAL_LIMIT,
    HEAVY_METAL_EQUAL_WEIGHT, SLOPE_BREAK, ASPECT_SCORES,
    PH_TOL_LOW, PH_TOL_HIGH,
)
def std_benefit(value, ideal):
    """越大越优指标标准化：以理想值为上限，按比例给分，饱和于 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, value / ideal))
def std_cost(value, extreme):
    """越小越优指标标准化：以极值为参考，线性折减至 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, 1.0 - value / extreme))
def std_mid_ph(value, lo, hi, tol_lo=PH_TOL_LOW,
    tol_hi=PH_TOL_HIGH):
    """适中型指标标准化（pH）：理想区间内取 1，区间外线性衰减。"""
    if value is None:
        return None
    if lo <= value <= hi:
        return 1.0
    if value > hi:
        return max(0.0, (tol_hi - value) / (tol_hi - hi))
    return max(0.0, (value - tol_lo) / (lo - tol_lo))
def std_slope(value, break_point=SLOPE_BREAK, upper=90.0):
    """坡度标准化。
    论文 4.2.2 节：以 25° 为陡坡重要分界。25° 及以下记满分 1，
    超过 25° 线性折减，90° 记 0。
    """
    if value is None:
        return None
    if value <= break_point:
        return 1.0
    return max(0.0, (upper - value) / (upper - break_point))
def aspect_score(aspect):
    """坡向离散赋值：阳坡 0.2 / 阴阳坡 0.6 / 阴坡 1.0（论文 4.2.2 节）。"""
    if aspect is None:
        return None
    return ASPECT_SCORES.get(aspect)
def pollution_score(heavy_metals):
    """污染指标：8 种重金属等权（各 0.125）合成。
    论文注：未检测 Cr、Zn 的，从矿山类型看未受污染，按无污染(=1)处理。
    heavy_metals : dict，键为金属名，值为含量 mg/kg；缺失键视为未检测。
    """
    total = 0.0
    for metal, extreme in HEAVY_METAL_LIMIT.items():
        value = heavy_metals.get(metal)
```

```
        total += std_cost(value, extreme) if value is not None else 1.0
    return total / len(HEAVY_METAL_LIMIT)
def standardize_unit(unit_type, data, weights=None):
    """对单个微地质单元的全部指标做标准化。
    参数
    ----
    unit_type : str，岩质边坡 / 碎石边坡 / 平台
    data : dict，实测数据，字段见接口契约：
        soil_ratio, thickness, ph, organic_matter,
    heavy_metals(dict),
        coverage, shannon, slope, aspect
    weights : dict，该单元类型的权重（app.utils.constants.DEFAULT_WEIGHTS 结构），
                未提供时用默认值。
    返回
    ----
    dict：{ 一级指标： { 二级指标： （实测值， 标准化值， 权重， 贡献） } }
    标准化值可能为 None，表示该指标不适用或方法待定。
    """
    from app.utils.constants import DEFAULT_WEIGHTS
    if weights is None:
        weights = DEFAULT_WEIGHTS[unit_type]
    l1_enabled = weights["l1"].keys()
    l2w = weights["l2"]
    result = {}
    # ---- 土壤指标 ----
    if "土壤指标" in l1_enabled:
        result["土壤指标"] = {
            "土石比": (data.get("soil_ratio"),
                       std_benefit(data.get("soil_ratio"),
    IDEAL_SOIL_RATIO),
                       l2w["土壤指标"]["土石比"]),
            "土层厚度": (data.get("thickness"),
                         std_benefit(data.get("thickness"),
    IDEAL_THICKNESS),
                         l2w["土壤指标"]["土层厚度"]),
            "pH": (data.get("ph"),
                   std_mid_ph(data.get("ph"),
    *IDEAL_PH_RANGE),
                   l2w["土壤指标"]["pH"]),
            "有机质": (data.get("organic_matter"),
                       std_benefit(data.get("organic_matter")
    , IDEAL_ORGANIC),
                       l2w["土壤指标"]["有机质"]),
            "污染指标": (data.get("heavy_metals") or {},
                         pollution_score(data.get("heavy_meta
    ls") or {}),
                         l2w["土壤指标"]["污染指标"]),
        }
```

```
    # ---- 植被指标 ----
    if "植被指标" in l1_enabled:
        result["植被指标"] = {
            "植被覆盖度": (data.get("coverage"),
                           std_benefit(data.get("coverage"),
IDEAL_COVERAGE),
                           l2w["植被指标"]["植被覆盖度"]),
            "生物多样性": (data.get("shannon"),
                           std_benefit(data.get("shannon"),
IDEAL_SHANNON),
                           l2w["植被指标"]["生物多样性"]),
        }
    # ---- 地形指标 ----
    if "地形指标" in l1_enabled:
        result["地形指标"] = {
            "坡度": (data.get("slope"),
                     std_slope(data.get("slope")),
                     l2w["地形指标"]["坡度"]),
            "坡向": (data.get("aspect"),
                     aspect_score(data.get("aspect")),
                     l2w["地形指标"]["坡向"]),
        }
    return result
# ==== app/services/evaluation_service.py ====
# -*- coding: utf-8 -*-
"""适宜性评价引擎（M8）。
按论文式(9) 实现：
- 单元级：F' = Σ（二级指标权重 × 该二级指标下三级指标标准化值）
- 矿山级：F = Σ(Si × F'i)，Si 为各微地质单元面积占比
- 等级判定按论文表 4。
"""
from app.utils.constants import DEFAULT_WEIGHTS, grade_of
from app.services import standardization as std
def score_unit(unit_type, data, weights=None):
    """计算单个微地质单元的评价因子 F'。
    返回
    ----
    dict：{ l1_scores: {一级指标: 得分}, unit_score: F', details:
    [...] }
    若某一级指标下存在"不适用"（标准化值为 None）的二级指标，
    则按其余项权重归一化后计算，保证单元得分始终有定义。
    """
    if weights is None:
        weights = DEFAULT_WEIGHTS[unit_type]
    std_result = std.standardize_unit(unit_type, data,
    weights)
    l1_weights = weights["l1"]
    l1_scores = {}
    details = []
    for l1_name, items in std_result.items():
```

```
        wsum = 0.0
        acc = 0.0
        for l2_name, (raw, std_value, w2) in items.items():
            if std_value is None:
                # 不适用项：不计入
                details.append({
                    "l1": l1_name, "l2": l2_name, "raw":
raw,
                    "std_value": None, "weight": w2,
"contribution": None,
                })
                continue
            contribution = std_value * w2
            acc += contribution
            wsum += w2
            details.append({
                "l1": l1_name, "l2": l2_name, "raw": raw,
                "std_value": std_value, "weight": w2,
                "contribution": contribution,
            })
        l1_scores[l1_name] = acc / wsum if wsum > 0 else 0.0
    unit_score = sum(l1_weights[k] * l1_scores[k] for k in
l1_scores)
    return {"l1_scores": l1_scores, "unit_score":
unit_score, "details": details}
def score_mine(units):
    """计算整个矿山的评价因子 F 与适宜性等级。
    参数
    ----
    units : list[dict]，每个元素：
        {unit_type, ratio, data}
        ratio 为该单元面积占比（0~1），data 为实测数据 dict。
    返回
    ----
    dict:{ final_score, grade, grade_info, unit_results }
    """
    total = 0.0
    unit_results = []
    for u in units:
        r = score_unit(u["unit_type"], u["data"])
        weighted = u["ratio"] * r["unit_score"]
        total += weighted
        unit_results.append({
            "unit_type": u["unit_type"],
            "ratio": u["ratio"],
            "unit_score": r["unit_score"],
            "weighted": weighted,
            "l1_scores": r["l1_scores"],
            "details": r["details"],
        })
```

```
    grade_info = grade_of(total)
    return {
        "final_score": total,
        "grade": grade_info["grade"],
        "grade_info": grade_info,
        "unit_results": unit_results,
    }
# ==== app/services/report_service.py ====
# -*- coding: utf-8 -*-
"""报告生成（M9）：生成 Word 评价报告。
依赖 python-docx。报告结构：
标题 → 图斑信息 → 评价结论 → 各微地质单元得分 → 完整得分明细 → 生成时间。
"""
import io
from datetime import datetime
from docx import Document
from docx.shared import Pt, Cm
from docx.oxml.ns import qn
from docx.enum.text import WD_ALIGN_PARAGRAPH
def _set_run(run, text=None, size=12, bold=False):
    if text is not None:
        run.text = text
    run.font.name = "Times New Roman"
    run.font.size = Pt(size)
    run.font.bold = bold
    # 中文字体
    run._element.rPr.rFonts.set(qn("w:eastAsia"), "宋体")
    return run
def _para(doc, text, size=12, bold=False, align=None,
    space_after=6):
    p = doc.add_paragraph()
    if align is not None:
        p.alignment = align
    p.paragraph_format.space_after = Pt(space_after)
    _set_run(p.add_run(), text, size, bold)
    return p
def _heading(doc, text, level=1):
    h = doc.add_heading(text, level=level)
    for run in h.runs:
        run.font.name = "Times New Roman"
        run._element.rPr.rFonts.set(qn("w:eastAsia"), "黑体")
    return h
def _table(doc, headers, rows, widths=None):
    t = doc.add_table(rows=1, cols=len(headers))
    t.style = "Table Grid"
    hdr = t.rows[0].cells
    for i, htext in enumerate(headers):
        _set_run(hdr[i].paragraphs[0].add_run(), htext,
    10.5, bold=True)
    for row in rows:
```

```
        cells = t.add_row().cells
        for i, val in enumerate(row):
            _set_run(cells[i].paragraphs[0].add_run(),
    str(val), 10.5)
    return t
def build_report(patch, mine_result):
    """生成评价报告 Word 文档，返回 BytesIO。
    参数
    ----
    patch : MinePatch，图斑对象
    mine_result : evaluation_service.score_mine 的返回 dict
    """
    doc = Document()
    # 页边距
    for section in doc.sections:
        section.top_margin = Cm(2.5)
        section.bottom_margin = Cm(2.5)
        section.left_margin = Cm(2.5)
        section.right_margin = Cm(2.5)
    _heading(doc, "历史遗留废弃矿山自然恢复适宜性评价报告", 0)
    _para(doc, "软件：历史遗留废弃矿山自然恢复适宜性评价系统 Ｖ1．0",
          size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER)
    _para(doc, "生成时间：%s" % datetime.now().strftime("%Y-%m-%d
%H：%M：%S"),
          size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER,
    space_after=12)
    # 图斑信息
    _heading(doc, "一、图斑基本信息", 1)
    info = [
        ("图斑名称", patch.name),
        ("所在行政区", patch.admin_region or "—"),
        ("图斑面积（公顷）", str(patch.area_ha) if patch.area_ha is
    not None else "—"),
        ("开采矿种", patch.mineral_type or "—"),
        ("开采方式", patch.mining_method or "—"),
    ]
    _table(doc, ["项目", "内容"], info)
    # 评价结论
    _heading(doc, "二、评价结论", 1)
    gi = mine_result["grade_info"]
    concl = [
        ("最终评价因子 F", "%.4f" % mine_result["final_score"]),
        ("适宜性等级", mine_result["grade"]),
        ("判定依据", gi["meaning"]),
        ("推荐措施", gi["measure"]),
    ]
    _table(doc, ["项目", "结果"], concl)
    # 各单元得分
    _heading(doc, "三、各微地质单元得分", 1)
    rows = []
```

```
    for u in mine_result["unit_results"]:
        rows.append((u["unit_type"], "%.2f%%" % (u["ratio"]
    * 100),
                     "%.4f" % u["unit_score"], "%.4f" %
    u["weighted"]))
    _table(doc, ["微地质单元", "面积占比", "单元评价因子 F'", "加权贡献"],
    rows)
    # 完整得分明细
    _heading(doc, "四、得分明细", 1)
    for u in mine_result["unit_results"]:
        _para(doc, "%s（占比 %.2f%%）" % (u["unit_type"],
    u["ratio"] * 100),
              size=11, bold=True, space_after=4)
        detail_rows = []
        for d in u["details"]:
            raw = "—" if d["raw"] is None else d["raw"]
            std = "—" if d["std_value"] is None else "%.4f"
    % d["std_value"]
            w = "%.4f" % d["weight"]
            c = "—" if d["contribution"] is None else
    "%.4f" % d["contribution"]
            detail_rows.append((d["l1"], d["l2"], raw, std,
    w, c))
        _table(doc, ["一级指标", "二级指标", "实测值", "标准化值", "权重",
    "贡献"],
               detail_rows)
        # 一级指标得分
        l1_row = []
        for k, v in u["l1_scores"].items():
            l1_row.append("%s=%.4f" % (k, v))
        _para(doc, "一级指标得分：" + "，".join(l1_row), size=10.5,
    space_after=10)
    buf = io.BytesIO()
    doc.save(buf)
    buf.seek(0)
    return buf
# ==== app/views/auth.py ====
# -*- coding: utf-8 -*-
"""M1 用户与权限管理 —— 登录、注销、权限控制。"""
from flask import Blueprint, render_template, redirect,
    url_for, request, flash
from flask_login import login_user, logout_user,
    login_required, current_user
from werkzeug.security import check_password_hash,
    generate_password_hash
from app.forms import LoginForm
from app.models import db, User
bp = Blueprint("auth", __name__)
@bp.route("/")
def index():
```

```
    if current_user.is_authenticated:                                  1051
        return render_template("auth/home.html")                       1052
    return redirect(url_for("auth.login"))                             1053
@bp.route("/login", methods=["GET", "POST"])                           1054
def login():                                                           1055
    if current_user.is_authenticated:                                  1056
        return redirect(url_for("auth.index"))                         1057
    form = LoginForm()                                                 1058
    if form.validate_on_submit():                                      1059
        user = User.query.filter_by(username=form.username.d           1060
    ata).first()                                                       1061
        if user and check_password_hash(user.password_hash,            1062
    form.password.data):                                               1063
            login_user(user, remember=form.remember.data)              1064
            flash("登录成功", "ok")                                    1065
            return redirect(url_for("auth.index"))                     1066
        flash("用户名或密码错误", "error")                             1067
    return render_template("auth/login.html", form=form)               1068
@bp.route("/logout")                                                   1069
@login_required                                                        1070
def logout():                                                          1071
    logout_user()                                                      1072
    flash("已退出登录", "ok")                                          1073
    return redirect(url_for("auth.login"))                             1074
@bp.route("/users")                                                    1075
@login_required                                                        1076
def users():                                                           1077
    """用户列表（管理员可管理）。"""                                   1078
    all_users = User.query.order_by(User.id).all()                     1079
    return render_template("auth/users.html",                          1080
    users=all_users)                                                   1081
@bp.route("/users/add", methods=["GET", "POST"])                       1082
@login_required                                                        1083
def add_user():                                                        1084
    if current_user.role != "admin":                                   1085
        flash("仅管理员可新增用户", "error")                           1086
        return redirect(url_for("auth.users"))                         1087
    if request.method == "POST":                                       1088
        username = request.form.get("username", "").strip()            1089
        password = request.form.get("password", "")                    1090
        role = request.form.get("role", "viewer")                      1091
        if not username or not password:                               1092
            flash("用户名和密码不能为空", "error")                     1093
            return redirect(url_for("auth.add_user"))                  1094
        if User.query.filter_by(username=username).first():            1095
            flash("用户名已存在", "error")                             1096
            return redirect(url_for("auth.add_user"))                  1097
        user = User(username=username,                                 1098
                    password_hash=generate_password_hash(pas           1099
    sword), role=role)                                                 1100
```

```
            db.session.add(user)                                    1101
            db.session.commit()                                     1102
            flash("用户已添加", "ok")                                1103
            return redirect(url_for("auth.users"))                  1104
        return render_template("auth/add_user.html")                1105
# ==== app/views/project.py ====                                    1106
# -*- coding: utf-8 -*-                                             1107
"""M2 项目与矿山档案 —— 项目、图斑、参照生态系统。"""                  1108
from flask import Blueprint, render_template, request,              1109
    redirect, url_for, flash                                        1110
from flask_login import login_required, current_user                1111
from app.forms import EmptyForm                                     1112
from app.models import db, Project, MinePatch,                      1113
    ReferenceEcosystem                                              1114
bp = Blueprint("project", __name__, url_prefix="/project")          1115
@bp.route("/")                                                      1116
@login_required                                                     1117
def index():                                                        1118
    projects = Project.query.order_by(Project.id.desc()).all1119
    ()                                                              1120
    return render_template("project/index.html",                    1121
    projects=projects,                                              1122
                                   form=EmptyForm())                1123
@bp.route("/add", methods=["GET", "POST"])                          1124
@login_required                                                     1125
def add():                                                          1126
    form = EmptyForm()                                              1127
    if request.method == "POST" and form.validate_on_submit(1128
    ):                                                              1129
        p = Project(                                                1130
            name=request.form.get("name", "").strip(),              1131
            code=request.form.get("code", "").strip() or            1132
    None,                                                           1133
            description=request.form.get("description",             1134
    "").strip() or None,                                            1135
            created_by=current_user.id,                             1136
        )                                                           1137
        if not p.name:                                              1138
            flash("项目名称不能为空", "error")                       1139
        else:                                                       1140
            db.session.add(p)                                       1141
            db.session.commit()                                     1142
            flash("项目已创建", "ok")                                1143
            return redirect(url_for("project.index"))               1144
    return render_template("project/edit.html", form=form,          1145
    project=None)                                                   1146
@bp.route("/<int:pid>/edit", methods=["GET", "POST"])               1147
@login_required                                                     1148
def edit(pid):                                                      1149
    p = Project.query.get_or_404(pid)                               1150
```

```
    form = EmptyForm()                                                  1151
    if request.method == "POST" and form.validate_on_submit(            1152
    ):                                                                  1153
        p.name = request.form.get("name", "").strip()                   1154
        p.code = request.form.get("code", "").strip() or                1155
    None                                                                1156
        p.description = request.form.get("description",                 1157
    "").strip() or None                                                 1158
        p.status = request.form.get("status", "进行中")                  1159
        db.session.commit()                                             1160
        flash("项目已保存", "ok")                                         1161
        return redirect(url_for("project.index"))                       1162
    return render_template("project/edit.html", form=form,              1163
    project=p)                                                          1164
@bp.route("/<int:pid>/delete", methods=["POST"])                        1165
@login_required                                                         1166
def delete(pid):                                                        1167
    p = Project.query.get_or_404(pid)                                   1168
    db.session.delete(p)                                                1169
    db.session.commit()                                                 1170
    flash("项目已删除", "ok")                                             1171
    return redirect(url_for("project.index"))                           1172
# ---------------------------- 矿山图斑 ----------------------------    1173
    ----                                                                1174
@bp.route("/<int:pid>/patches")                                         1175
@login_required                                                         1176
def patches(pid):                                                       1177
    p = Project.query.get_or_404(pid)                                   1178
    return render_template("project/patches.html",                      1179
    project=p,                                                          1180
                                form=EmptyForm())                       1181
@bp.route("/patch/<int:patch_id>/delete", methods=["POST"])             1182
@login_required                                                         1183
def patch_delete(patch_id):                                             1184
    """删除矿山图斑，及其微地质单元与指标数据。"""                                1185
    from app.models import MicroUnit, IndicatorData,                    1186
    EvaluationResult, \                                                 1187
        ScoreDetail, ConstraintCheck, ReferenceEcosystem                1188
    patch = MinePatch.query.get_or_404(patch_id)                        1189
    pid = patch.project_id                                              1190
    unit_ids = [u.id for u in MicroUnit.query.filter_by(patc            1191
    h_id=patch.id).all()]                                               1192
    if unit_ids:                                                        1193
        IndicatorData.query.filter(IndicatorData.unit_id.in_            1194
    (unit_ids)).delete(                                                 1195
            synchronize_session=False)                                  1196
    for r in EvaluationResult.query.filter_by(patch_id=patch            1197
    .id).all():                                                         1198
        ScoreDetail.query.filter_by(result_id=r.id).delete(             1199
            synchronize_session=False)                                  1200
```

```
    EvaluationResult.query.filter_by(patch_id=patch.id).dele
    te(
        synchronize_session=False)
    ConstraintCheck.query.filter_by(patch_id=patch.id).delet
    e(
        synchronize_session=False)
    MicroUnit.query.filter_by(patch_id=patch.id).delete(sync
    hronize_session=False)
    ReferenceEcosystem.query.filter_by(patch_id=patch.id).de
    lete(
        synchronize_session=False)
    db.session.delete(patch)
    db.session.commit()
    flash("图斑「%s」及其数据已删除" % patch.name, "ok")
    return redirect(url_for("project.patches", pid=pid))
@bp.route("/<int:pid>/patch/add", methods=["GET", "POST"])
@login_required
def patch_add(pid):
    p = Project.query.get_or_404(pid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        patch = MinePatch(
            project_id=p.id,
            name=request.form.get("name", "").strip(),
            admin_region=fv("admin_region"),
            area_ha=fv("area_ha", float),
            mineral_type=fv("mineral_type"),
            mining_method=fv("mining_method"),
            stop_years=fv("stop_years", float),
            responsible_entity_missing=bool(request.form.get
    ("responsible_entity_missing")),
        )
        if not patch.name:
            flash("图斑名称不能为空", "error")
        else:
            db.session.add(patch)
            db.session.flush()
            # 参照生态系统默认取论文值
            db.session.add(ReferenceEcosystem(
                patch_id=patch.id, coverage_benchmark=75.0,
    shannon_benchmark=0.7))
            db.session.commit()
            flash("图斑已创建", "ok")
            return redirect(url_for("project.patches",
    pid=p.id))
    return render_template("project/patch_edit.html",
```

```
    form=form, project=p, patch=None)
@bp.route("/patch/<int:patch_id>/edit", methods=["GET",
    "POST"])
@login_required
def patch_edit(patch_id):
    patch = MinePatch.query.get_or_404(patch_id)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        patch.name = request.form.get("name", "").strip()
        patch.admin_region = fv("admin_region")
        patch.area_ha = fv("area_ha", float)
        patch.mineral_type = fv("mineral_type")
        patch.mining_method = fv("mining_method")
        patch.stop_years = fv("stop_years", float)
        patch.responsible_entity_missing =
    bool(request.form.get("responsible_entity_missing"))
        # 参照生态系统
        ref = ReferenceEcosystem.query.filter_by(patch_id=pa
    tch.id).first()
        if ref is None:
            ref = ReferenceEcosystem(patch_id=patch.id)
            db.session.add(ref)
        ref.coverage_benchmark = fv("coverage_benchmark",
    float) or 75.0
        ref.shannon_benchmark = fv("shannon_benchmark",
    float) or 0.7
        ref.dominant_species = fv("dominant_species")
        db.session.commit()
        flash("图斑已保存", "ok")
        return redirect(url_for("project.patches",
    pid=patch.project_id))
    return render_template("project/patch_edit.html",
    form=form,
                           project=patch.project,
    patch=patch)
# ==== app/views/unit.py ====
# -*- coding: utf-8 -*-
"""M5 微地质单元管理 —— 岩质边坡 / 碎石边坡 / 平台的划分与面积占比。"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, MinePatch, MicroUnit,
    IndicatorData
from app.utils.constants import UNIT_TYPES
bp = Blueprint("unit", __name__, url_prefix="/unit")
```

```
def recalc_ratio(patch_id):
    """按面积重算该图斑各单元面积占比。"""
    units = MicroUnit.query.filter_by(patch_id=patch_id).all()
    total = sum(u.area or 0.0 for u in units)
    if total > 0:
        for u in units:
            u.ratio = round((u.area or 0.0) / total, 6)
    return total
@bp.route("/<int:patch_id>")
@login_required
def index(patch_id):
    patch = MinePatch.query.get_or_404(patch_id)
    units = MicroUnit.query.filter_by(patch_id=patch_id).all()
    total = sum(u.area or 0.0 for u in units)
    ratio_sum = sum(u.ratio or 0.0 for u in units)
    return render_template("unit/index.html", patch=patch,
    units=units,
                           total_area=total,
    ratio_sum=ratio_sum,
                           unit_types=UNIT_TYPES,
    form=EmptyForm())
@bp.route("/<int:patch_id>/add", methods=["POST"])
@login_required
def add(patch_id):
    patch = MinePatch.query.get_or_404(patch_id)
    form = EmptyForm()
    if form.validate_on_submit():
        unit_type = request.form.get("unit_type")
        area = request.form.get("area", "").strip()
        u = MicroUnit(patch_id=patch.id,
    unit_type=unit_type,
                      area=float(area) if area else None,
                      description=request.form.get("descript
    ion", "").strip() or None)
        db.session.add(u)
        db.session.flush()
        recalc_ratio(patch.id)
        db.session.commit()
        flash("微地质单元已添加", "ok")
    return redirect(url_for("unit.index",
    patch_id=patch_id))
@bp.route("/edit/<int:unit_id>", methods=["GET", "POST"])
@login_required
def edit(unit_id):
    """单个微地质单元的编辑页。"""
    u = MicroUnit.query.get_or_404(unit_id)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
```

```
    ):                                                                    1351
        u.unit_type = request.form.get("unit_type",                       1352
    u.unit_type)                                                          1353
        area = request.form.get("area", "").strip()                       1354
        u.area = float(area) if area else None                            1355
        u.description = request.form.get("description",                   1356
    "").strip() or None                                                   1357
        recalc_ratio(u.patch_id)                                          1358
        db.session.commit()                                               1359
        flash("微地质单元已保存", "ok")                                    1360
        return redirect(url_for("unit.index",                             1361
    patch_id=u.patch_id))                                                 1362
    return render_template("unit/edit.html", unit=u,                      1363
    patch=u.patch,                                                        1364
                                unit_types=UNIT_TYPES, form=form)         1365
@bp.route("/delete/<int:unit_id>", methods=["POST"])                      1366
@login_required                                                           1367
def delete(unit_id):                                                      1368
    u = MicroUnit.query.get_or_404(unit_id)                               1369
    pid = u.patch_id                                                      1370
    name = u.unit_type                                                    1371
    # 一并删除该单元的指标数据，避免残留                                  1372
    IndicatorData.query.filter_by(unit_id=u.id).delete(synch              1373
    ronize_session=False)                                                 1374
    db.session.delete(u)                                                  1375
    db.session.flush()                                                    1376
    recalc_ratio(pid)                                                     1377
    db.session.commit()                                                   1378
    flash("微地质单元「%s」已删除" % name, "ok")                          1379
    return redirect(url_for("unit.index", patch_id=pid))                  1380
# ==== app/views/standard.py ====                                         1381
# -*- coding: utf-8 -*-                                                   1382
"""M3 政策与标准知识库 —— 标准 / 政策文件的结构化规则与省际口径比对。"""  1383
from flask import Blueprint, render_template, request,                    1384
    redirect, url_for, flash                                              1385
from flask_login import login_required                                    1386
from app.forms import EmptyForm                                           1387
from app.models import db, Standard, StandardRule                         1388
bp = Blueprint("standard", __name__, url_prefix="/standard")              1389
@bp.route("/")                                                            1390
@login_required                                                           1391
def index():                                                              1392
    standards = Standard.query.order_by(Standard.category,                1393
    Standard.province).all()                                              1394
    local = [s for s in standards if s.category == "地方标准"]            1395
    policy = [s for s in standards if s.category != "地方标准"]           1396
    return render_template("standard/index.html",                         1397
    local=local, policy=policy,                                           1398
                                form=EmptyForm())                         1399
@bp.route("/add", methods=["GET", "POST"])                                1400
```

```
@login_required
def add():
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        s = Standard(
            province=request.form.get("province",
    "").strip(),
            name=request.form.get("name", "").strip(),
            doc_number=request.form.get("doc_number",
    "").strip() or None,
            publish_date=request.form.get("publish_date",
    "").strip() or None,
            implement_date=request.form.get("implement_date"
    , "").strip() or None,
            category=request.form.get("category", "地方标准"),
            summary=request.form.get("summary", "").strip()
    or None,
        )
        if not s.province or not s.name:
            flash("省份与名称不能为空", "error")
        else:
            db.session.add(s)
            db.session.commit()
            flash("标准已添加", "ok")
            return redirect(url_for("standard.index"))
    return render_template("standard/edit.html", form=form,
    standard=None)
@bp.route("/<int:sid>/edit", methods=["GET", "POST"])
@login_required
def edit(sid):
    s = Standard.query.get_or_404(sid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        s.province = request.form.get("province",
    "").strip()
        s.name = request.form.get("name", "").strip()
        s.doc_number = request.form.get("doc_number",
    "").strip() or None
        s.publish_date = request.form.get("publish_date",
    "").strip() or None
        s.implement_date = request.form.get("implement_date"
    , "").strip() or None
        s.category = request.form.get("category", "地方标准")
        s.summary = request.form.get("summary", "").strip()
    or None
        db.session.commit()
        flash("已保存", "ok")
        return redirect(url_for("standard.rules", sid=s.id))
```

```
    return render_template("standard/edit.html", form=form,
    standard=s)
@bp.route("/<int:sid>/delete", methods=["POST"])
@login_required
def delete(sid):
    s = Standard.query.get_or_404(sid)
    StandardRule.query.filter_by(standard_id=sid).delete()
    db.session.delete(s)
    db.session.commit()
    flash("已删除", "ok")
    return redirect(url_for("standard.index"))
@bp.route("/<int:sid>/rules", methods=["GET", "POST"])
@login_required
def rules(sid):
    s = Standard.query.get_or_404(sid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        r = StandardRule(
            standard_id=s.id,
            item=request.form.get("item", "").strip(),
            operator=request.form.get("operator", ">="),
            threshold=request.form.get("threshold",
    "").strip() or None,
            condition_type=request.form.get("condition_type"
    , "必要条件"),
            region=request.form.get("region", "").strip()
    or None,
            note=request.form.get("note", "").strip() or
    None,
        )
        if not r.item:
            flash("判定项不能为空", "error")
        else:
            db.session.add(r)
            db.session.commit()
            flash("规则已添加", "ok")
        return redirect(url_for("standard.rules", sid=s.id))
    rs = StandardRule.query.filter_by(standard_id=s.id).all(
    )
    return render_template("standard/rules.html",
    form=form, standard=s, rules=rs)
@bp.route("/rule/<int:rid>/delete", methods=["POST"])
@login_required
def rule_delete(rid):
    r = StandardRule.query.get_or_404(rid)
    sid = r.standard_id
    db.session.delete(r)
    db.session.commit()
    flash("规则已删除", "ok")
```

```
      <fieldset style="border:1px solid                                    3301
      var(--line);border-radius:6px;padding:10px                           3302
      14px;margin:0 0 10px;">                                              3303
        <legend style="padding:0 6px;font-weight:600;">{{                  3304
      name }}</legend>                                                     3305
        <div class="actions">                                              3306
          <label class="checkbox">                                         3307
            <input type="radio" name="{{ field }}" value="1"               3308
                   {% if val is none or val %}checked{% endif              3309
      %}>                                                                  3310
            <span>通过</span>                                              3311
          </label>                                                         3312
          <label class="checkbox">                                         3313
            <input type="radio" name="{{ field }}" value="0"               3314
                   {% if val is not none and not val                       3315
      %}checked{% endif %}>                                                3316
            <span>不通过</span>                                            3317
          </label>                                                         3318
        </div>                                                             3319
      </fieldset>                                                          3320
      {% endfor %}                                                         3321
      <p class="actions" style="margin-top:16px;">                         3322
        <button type="submit" class="btn">保存判定</button>                3323
        <a class="btn btn--ghost" href="/constraint/">返回</a>             3324
      </p>                                                                 3325
    </form>                                                                3326
    {% if record %}                                                        3327
    <div style="margin-top:18px;padding:14px;border-radius:6px3328
      ;border-left:4px solid;                                              3329
                  {% if record.passed %}background:var(--ok-soft3330
      );border-left-color:var(--ok);                                       3331
                  {% else %}background:var(--danger-soft);border3332
      -left-color:var(--danger);{% endif %}">                              3333
      <p style="margin:0 0 4px;font-weight:600;">                          3334
        判定结论：                                                         3335
        {% if record.passed %}                                             3336
        <span class="ok">通过，可进入定量评价</span>                       3337
        {% else %}                                                         3338
        <span class="warn">不通过 —— 建议生态重建或人工辅助</span>         3339
        {% endif %}                                                        3340
      </p>                                                                 3341
      <p class="muted" style="margin:0;">{{ record.reason                  3342
      }}</p>                                                               3343
    </div>                                                                 3344
    {% endif %}                                                            3345
</div>                                                                     3346
{% endblock %}                                                             3347
# ==== app/templates/ahp/index.html ====                                   3348
{% extends "base.html" %}                                                  3349
{% block title %}AHP 权重计算{% endblock %}                                3350
```

```
{% block content %}
<div class="card" style="max-width:720px;">
  <h2>AHP 权重计算 · 选择指标层级</h2>
  <p class="muted">
    选择微地质单元类型与指标层级后，系统按论文表 3 确定判断矩阵的规模。
    岩质边坡不设土壤指标、平台不设地形指标，系统会相应调整矩阵所包含的指标。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="{{ form.unit_type.id }}">微地质单元类型</label>
      {{ form.unit_type(id=form.unit_type.id) }}
    </div>
    <div class="field">
      <label class="field__label" for="{{ form.level.id }}">指标层级</label>
      {{ form.level(id=form.level.id) }}
    </div>
    <div class="field">
      <label class="field__label" for="{{ form.l1.id }}">
        一级指标（层级选「二级指标」时需选择）
      </label>
      {{ form.l1(id=form.l1.id) }}
    </div>
    <p class="actions" style="margin-top:16px;">
      {{ form.submit(class_="btn") }}
    </p>
  </form>
</div>
{% endblock %}
# ==== app/templates/ahp/matrix.html ====
{% extends "base.html" %}
{% block title %}判断矩阵录入{% endblock %}
{% block content %}
<div class="card">
  <h2>
    判断矩阵录入
    · {{ unit_type }}
    · {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{% endif %}
  </h2>
  <p class="muted">
    采用 1–9 标度：1 同等重要、3 稍微重要、5 明显重要、7 强烈重要、9 极端重要，
    2、4、6、8 为相邻判断的中间值。只需录入上三角，对角线为 1，下三角自动取倒数。
    矩阵已按论文表 3 的权重预填。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    <table style="width:auto;">
```

```
    <thead>                                                                3401
      <tr>                                                                 3402
        <th></th>                                                          3403
        {% for name in names %}<th scope="col">{{ name                     3404
  }}</th>{% endfor %}                                                      3405
      </tr>                                                                3406
    </thead>                                                               3407
    <tbody>                                                                3408
      {% for i in range(names|length) %}                                   3409
      <tr>                                                                 3410
        <th scope="row">{{ names[i] }}</th>                                3411
        {% for j in range(names|length) %}                                 3412
          {% if i == j %}                                                  3413
          <td class="muted num">1</td>                                     3414
          {% elif i < j %}                                                 3415
          <td>                                                             3416
            <label class="muted" style="position:absolute;                 3417
  left:-9999px;"                                                           3418
                   for="m-{{ i }}-{{ j }}">{{ names[i] }}                  3419
  相对 {{ names[j] }} 的重要性</label>                                     3420
            <input id="m-{{ i }}-{{ j }}" type="number"                    3421
  step="0.01" min="0.01"                                                   3422
                   name="m_{{ i }}_{{ j }}" value="{{                      3423
  default[i][j] }}">                                                       3424
          </td>                                                            3425
          {% else %}                                                       3426
          <td class="muted num">{{ default[i][j] }}</td>                   3427
          {% endif %}                                                      3428
        {% endfor %}                                                       3429
      </tr>                                                                3430
      {% endfor %}                                                         3431
    </tbody>                                                               3432
  </table>                                                                 3433
  <p class="actions">                                                      3434
    <button type="submit" class="btn">计算权重</button>                    3435
    <a class="btn btn--ghost" href="/ahp/">重新选择</a>                    3436
  </p>                                                                     3437
</form>                                                                    3438
{% if result %}                                                            3439
<hr style="margin:22px 0;border:none;border-top:1px solid                  3440
  var(--line);">                                                           3441
<h3>计算结果</h3>                                                          3442
<table style="width:auto;">                                                3443
  <thead>                                                                  3444
    <tr>                                                                   3445
      <th scope="col">指标</th>                                            3446
      {% for name in names %}<th scope="col">{{ name                       3447
  }}</th>{% endfor %}                                                      3448
    </tr>                                                                  3449
  </thead>                                                                 3450
```

```
      <tbody>
        <tr>
          <th scope="row">权重</th>
          {% for w in result.weights %}<td class="mono">{{ w }}</td>{% endfor %}
        </tr>
      </tbody>
    </table>
    <div class="stat-row">
      <div>
        <div class="stat__label">最大特征值 λmax</div>
        <div class="stat__value mono">{{ result.lambda_max }}</div>
      </div>
      <div>
        <div class="stat__label">一致性指标 CI</div>
        <div class="stat__value mono">{{ "%.4f"|format(result.ci) }}</div>
      </div>
      <div>
        <div class="stat__label">一致性比例 CR</div>
        <div class="stat__value mono">{{ "%.4f"|format(result.cr) }}</div>
      </div>
      <div>
        <div class="stat__label">一致性检验（要求 CR &lt; 0.1）</div>
        <div class="stat__value">
          {% if result.consistent %}<span class="ok">通过</span>
          {% else %}<span class="warn">不通过</span>{% endif %}
        </div>
      </div>
    </div>
    {% if not result.consistent and result.most_inconsistent
      %}
    <p style="margin-top:14px;padding:12px;border-radius:6px;b
      ackground:var(--danger-soft);
                border-left:4px solid var(--danger);">
      <strong class="warn">建议优先调整：</strong>
      第 {{ result.most_inconsistent.i + 1 }} 行、第 {{
      result.most_inconsistent.j + 1 }} 列的比较
      （当前 <span class="mono num">{{ result.most_inconsistent.v
      alue|round(3) }}</span>，
      理想约 <span class="mono num">{{ result.most_inconsistent.e
      xpected|round(3) }}</span>）
    </p>
    {% endif %}
    {% endif %}
</div>
{% endblock %}
# ==== app/templates/indicator/index.html ====
```

```
{% extends "base.html" %}
{% block title %}指标数据采集{% endblock %}
{% block content %}
<div class="card">
  <h2>指标数据采集与管理</h2>
  <p class="muted">
    按微地质单元组织录入。系统按单元类型自动决定应录入的指标：
    岩质边坡不启用土壤指标、平台不启用地形指标。
  </p>
  {% if not patches %}
  <p class="muted">
    暂无矿山图斑。请先在「项目与档案」中建立图斑，
    或运行 <span class="mono">python seed_demo.py</span>
    载入论文案例。
  </p>
  {% else %}
  {% for p in patches %}
  <h3>{{ p.name }}</h3>
  {% if not p.units.count() %}
  <p class="muted">该图斑尚未划分微地质单元，请先到「微地质单元」页面划分。</p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">微地质单元</th>
        <th scope="col">面积占比</th>
        <th scope="col">数据状态</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for u in p.units %}
      <tr>
        <td>{{ u.unit_type }}</td>
        <td class="num">
          {% if u.ratio is not none %}{{
    "%.2f%%"|format(u.ratio * 100) }}{% else %}—{% endif %}
        </td>
        <td>
          {% if u.indicator_data %}
          <span class="tag tag--ok">已录入</span>
          {% else %}
          <span class="tag tag--muted">未录入</span>
          {% endif %}
        </td>
        <td><a href="/indicator/{{ u.id }}">录入 / 编辑</a></td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
```

```
  {% endif %}
  {% endfor %}
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/indicator/edit.html ====
{% extends "base.html" %}
{% block title %}指标数据录入 · {{ unit.unit_type }}{% endblock %}
{% block content %}
<div class="card" style="max-width:820px;">
  <h2>指标数据录入 · {{ unit.unit_type }}</h2>
  <p class="muted">
    输入框留空表示该指标「不适用」，与「实测值为 0」含义不同，计算时会被跳过。
    岩质边坡不设土壤指标、平台不设地形指标，系统已按单元类型隐藏不相关字段。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    {% for group_name, fields, has_hm in groups %}
    <h3>{{ group_name }}</h3>
    {% for fname, cname, unit_ in fields %}
    <div class="field">
      <label class="field__label" for="f-{{ fname }}">
        {{ cname }}{% if unit_ %} ({{ unit_ }}){% endif %}
      </label>
      {% if fname == "aspect" %}
      <select id="f-{{ fname }}" name="aspect">
        <option value="">— 不适用 —</option>
        {% for a in aspect_types %}
        <option value="{{ a }}" {% if current.get("aspect")
    == a %}selected{% endif %}>{{ a }}</option>
        {% endfor %}
      </select>
      {% else %}
      <input id="f-{{ fname }}" type="number" step="any"
    name="{{ fname }}"
             value="{{ current.get(fname) if
    current.get(fname) is not none else '' }}">
      {% endif %}
    </div>
    {% endfor %}
    {% endfor %}
    {% for group_name, fields, has_hm in groups %}
    {% if has_hm %}
    <h3>重金属含量（mg/kg）</h3>
    <p class="muted">未检测的金属可留空，系统按论文注释视为未受污染处理。</p>
    <table style="width:auto;">
      <thead>
        <tr>
          {% for m in heavy_metals %}<th scope="col">{{ m
    }}</th>{% endfor %}
```

```
            </tr>                                                                3601
          </thead>                                                               3602
          <tbody>                                                                3603
            <tr>                                                                 3604
              {% for m in heavy_metals %}                                        3605
              <td>                                                               3606
                <label class="muted" style="position:absolute;                   3607
    left:-9999px;"                                                               3608
                       for="hm-{{ m }}">{{ m }} 含量</label>                     3609
                <input id="hm-{{ m }}" type="number"                             3610
    step="any" name="hm_{{ m }}"                                                 3611
                       value="{{ current.get('heavy',                            3612
    {}).get(m) if current.get('heavy', {}).get(m) is not                         3613
    none else '' }}">                                                            3614
              </td>                                                              3615
              {% endfor %}                                                       3616
            </tr>                                                                3617
          </tbody>                                                               3618
        </table>                                                                 3619
        {% endif %}                                                              3620
    {% endfor %}                                                                 3621
    <p class="actions" style="margin-top:18px;">                                 3622
      <button type="submit" class="btn">保存</button>                            3623
      <a class="btn btn--ghost" href="/indicator/">返回</a>                      3624
    </p>                                                                         3625
  </form>                                                                        3626
</div>                                                                           3627
{% endblock %}                                                                   3628
# ==== app/templates/evaluate/index.html ====                                    3629
{% extends "base.html" %}                                                        3630
{% block title %}适宜性评价{% endblock %}                                        3631
{% block content %}                                                              3632
<div class="card">                                                               3633
  <h2>适宜性定量评价</h2>                                                        3634
  <p class="muted">                                                              3635
    系统按论文式（8） 对各项指标做标准化，按式（9） 分单元赋分并按面积占比加权合成，  3636
    最后按论文表 4 判定适宜性等级。                                              3637
  </p>                                                                           3638
  {% if not patches %}                                                           3639
  <p class="muted">暂无矿山图斑。请先在「项目与档案」中建立图斑并录入指标数据。</p>  3640
  {% else %}                                                                     3641
  <table>                                                                        3642
    <thead>                                                                      3643
      <tr>                                                                       3644
        <th scope="col">矿山图斑</th>                                            3645
        <th scope="col">最近评价结果</th>                                        3646
        <th scope="col">操作</th>                                                3647
      </tr>                                                                      3648
    </thead>                                                                     3649
    <tbody>                                                                      3650
```

```
        {% for p in patches %}
        <tr>
          <td style="text-align:left;">{{ p.name }}</td>
          <td>
            {% if latest.get(p.id) %}
            F = <span class="mono num">{{ latest[p.id].final_score }}</span>,
            <span class="tag tag--ok">{{ latest[p.id].grade }}</span>
            {% else %}
            <span class="tag tag--muted">未评价</span>
            {% endif %}
          </td>
          <td class="nowrap">
            <a href="/evaluate/{{ p.id }}">执行评价</a>
            {% if latest.get(p.id) %}
             | <a href="/evaluate/result/{{ latest[p.id].id }}">查看结果</a>
            {% endif %}
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
    {% endif %}
</div>
{% endblock %}
# ==== app/templates/evaluate/run.html ====
{% extends "base.html" %}
{% block title %}执行评价 · {{ patch.name }}{% endblock %}
{% block content %}
<div class="card" style="max-width:640px;">
    <h2>执行适宜性评价 · {{ patch.name }}</h2>
    <p class="muted">
      系统将对图斑内各微地质单元做指标标准化、分单元赋分、面积加权合成，
      并判定适宜性等级。评价结果会保存，可随时回看。
    </p>
    <form method="post">
      {{ form.hidden_tag() }}
      <p class="actions" style="margin-top:16px;">
        <button type="submit" class="btn">开始评价</button>
        <a class="btn btn--ghost" href="/evaluate/">返回</a>
      </p>
    </form>
</div>
{% endblock %}
# ==== app/templates/evaluate/result.html ====
{% extends "base.html" %}
{% block title %}评价结果{% endblock %}
{% block content %}
```

```
<div class="card">
  <h2>适宜性评价结果 · {{ patch.name if patch else "" }}</h2>
  <div class="stat-row">
    <div>
      <div class="stat__label">最终评价因子 F</div>
      <div class="stat__value mono">{{ r.final_score
    }}</div>
    </div>
    <div>
      <div class="stat__label">适宜性等级</div>
      <div class="stat__value ok">{{ r.grade }}</div>
    </div>
    <div style="flex:1;min-width:240px;">
      <div class="stat__label">推荐措施</div>
      <div style="font-size:15px;">{{ r.measures }}</div>
    </div>
  </div>
  {% if unit_scores %}
  <h3>各微地质单元得分</h3>
  <table style="max-width:720px;">
    <thead>
      <tr>
        <th scope="col">微地质单元</th>
        <th scope="col">面积占比</th>
        <th scope="col">单元评价因子 F'</th>
        <th scope="col">加权贡献</th>
      </tr>
    </thead>
    <tbody>
      {% for u in unit_scores %}
      <tr>
        <td>{{ u.unit_type }}</td>
        <td class="num">{{ "%.2f%%"|format(u.ratio * 100)
    }}</td>
        <td class="mono">{{ u.unit_score }}</td>
        <td class="mono">{{ "%.4f"|format(u.ratio *
    u.unit_score) }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
  <p class="actions" style="margin-top:18px;">
    <a class="btn btn--ghost" href="/report/{{ r.patch_id
    }}">查看得分明细</a>
    <a class="btn btn--ghost" href="/evaluate/">返回评价列表</a>
  </p>
</div>
{% endblock %}
# ==== app/templates/report/index.html ====
```

```
{% extends "base.html" %}
{% block title %}报告输出{% endblock %}
{% block content %}
<div class="card">
  <h2>报告与结果输出</h2>
  <p class="muted">可查看逐层得分明细，并导出规范的评价报告文档。</p>
  {% if not patches %}
  <p class="muted">暂无矿山图斑。请先在「项目与档案」中建立图斑并完成评价。</p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">矿山图斑</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for p in patches %}
      <tr>
        <td style="text-align:left;">{{ p.name }}</td>
        <td class="nowrap">
          <a href="/report/{{ p.id }}">查看得分明细</a>
           | <a href="/report/{{ p.id }}/word">导出
    Word 报告</a>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/report/detail.html ====
{% extends "base.html" %}
{% block title %}得分明细 · {{ patch.name }}{% endblock %}
{% block content %}
<div class="card">
  <h2>得分明细 · {{ patch.name }}</h2>
  <div class="stat-row">
    <div>
      <div class="stat__label">最终评价因子 F</div>
      <div class="stat__value mono">{{
    "%.4f"|format(result.final_score) }}</div>
    </div>
    <div>
      <div class="stat__label">适宜性等级</div>
      <div class="stat__value ok">{{ result.grade }}</div>
    </div>
    <div style="flex:1;min-width:240px;">
      <div class="stat__label">推荐措施</div>
```

```
        <div style="font-size:15px;">{{
      result.grade_info.measure }}</div>
      </div>
    </div>
    <p class="actions" style="margin-top:16px;">
      <a class="btn" href="/report/{{ patch.id }}/word">导出
      Word 报告</a>
      <a class="btn btn--ghost" href="/report/">返回</a>
    </p>
</div>
{% for u in result.unit_results %}
<div class="card">
    <h2>{{ u.unit_type }} · 占比 {{ "%.2f%%"|format(u.ratio *
      100) }} · 单元得分 {{ "%.4f"|format(u.unit_score) }}</h2>
    <table>
      <thead>
        <tr>
          <th scope="col">一级指标</th>
          <th scope="col">二级指标</th>
          <th scope="col">实测值</th>
          <th scope="col">标准化值</th>
          <th scope="col">权重</th>
          <th scope="col">贡献</th>
        </tr>
      </thead>
      <tbody>
        {% for d in u.details %}
        <tr>
          <td>{{ d.l1 }}</td>
          <td style="text-align:left;">{{ d.l2 }}</td>
          <td class="mono" style="text-align:left;">{{ "—" if
      d.raw is none else d.raw }}</td>
          <td class="mono">{{ "—" if d.std_value is none else
      "%.4f"|format(d.std_value) }}</td>
          <td class="mono">{{ "%.4f"|format(d.weight) }}</td>
          <td class="mono">{{ "—" if d.contribution is none
      else "%.4f"|format(d.contribution) }}</td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
    <p class="muted" style="margin:0;">
      一级指标得分：
      {% for k, v in u.l1_scores.items() %}{{ k }} = <span
      class="num">{{ "%.4f"|format(v) }}</span>{% if not
      loop.last %};{% endif %}{% endfor %}
    </p>
</div>
{% endfor %}
{% endblock %}
```

```
# ==== seed_demo.py ====
# -*- coding: utf-8 -*-
"""内置玉田县案例：把论文第 5 节案例数据写入数据库。
用于功能验证与操作演示。数据逐字取自论文表 5 与第 5 节。
运行：python seed_demo.py（幂等：已存在则先清空重建）
"""
import json
import sys
sys.stdout.reconfigure(encoding="utf-8")
from app import create_app
from app.models import (db, User, Project, MinePatch,
    ReferenceEcosystem,
                        MicroUnit, IndicatorData, Standard,
    StandardRule)
app = create_app()
# 论文表 1：11 项地方标准与政策文件
STANDARDS = [
    dict(category="地方标准", province="辽宁",
    name="矿山废弃地植被自然恢复技术规程",
        doc_number="DB21/T 3702—2023",
    publish_date="2023-03-30",
        implement_date="2023-04-30",
        summary="首个正式发布的矿山自然恢复地方标准；提出必要条件和充分条件",
        rules=[("植被覆盖度", ">=", "50", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("无水土污染", "==", "是", "必要条件", None),
               ("与参照系差距", "<", "20", "充分条件", "%")]),
    dict(category="地方标准", province="湖北",
    name="废弃露天矿山生态环境自然恢复标准",
        doc_number="DB42/T 2421—2025",
    publish_date="2025-08-20",
        implement_date="2025-10-20",
        summary="结合矿山生态需求进行自然恢复适宜性分级判断",
        rules=[("自然恢复适宜性分级", "==", "是", "必要条件", None)]),
    dict(category="地方标准", province="内蒙古",
    name="历史遗留废弃矿山自然恢复判定规范",
        doc_number="DB15/T 4280—2026",
    publish_date="2026-01-30",
        implement_date="2026-03-01",
        summary="调查内容包含参照生态系统调查",
        rules=[("地质灾害发育程度", "<=", "弱发育", "必要条件", None),
               ("地形地貌与周边协调", "==", "是", "必要条件", None),
               ("有利于植被恢复的表层土壤", "==", "是", "必要条件", None),
               ("已形成稳定植被覆盖", "==", "是", "必要条件", None)]),
    dict(category="地方标准", province="江西",
    name="废弃矿山自然恢复技术规范",
        doc_number="DB36/T 2221—2026",
    publish_date="2026-02-02",
        implement_date="2026-08-01",
        summary="对参照生态系统进行了要求；提出自然恢复适宜性指数",
```

```
        rules=[("参照生态系统调查", "==", "是", "必要条件", None),
               ("自然恢复适宜性指数", ">=", "阈值", "充分条件", None)]),
    dict(category="地方标准", province="宁夏", name="矿山自然恢复认定规程",
        doc_number=None, publish_date="2026-06-18",
    implement_date=None,
        summary="分区差异化：三个一级生态修复分区及 22 个二级分区",
        rules=[("无地质灾害风险", "==", "是", "必要条件", None),
               ("植被覆盖度趋势", "==", "稳定或正增长", "必要条件", None),
               ("与参照系差距", "<", "20", "充分条件", "%")]),
    dict(category="政策文件", province="安徽",
        name="安徽省自然资源厅关于规范废弃露天矿山采取自然恢复方式进行治理的通知",
        doc_number="皖自然资修函[2020]34号",
    publish_date="2020-03-03", implement_date=None,
        summary="自然恢复条件：责任主体为政府、无地质灾害、无水土污染、不在三线三边、水土条件良好",
        rules=[("责任主体", "==", "政府", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("无水土污染", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="山东",
        name="山东省自然资源厅关于进一步规范历史遗留废弃露天开采矿山采取自然恢复方式进行治理的通知",
        doc_number="鲁自然资字[2022]104号",
    publish_date="2022-08-02", implement_date=None,
        summary="条件与安徽省相同，另增加自然恢复认定标准",
        rules=[("无地质灾害", "==", "是", "必要条件", None),
               ("无水土污染", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="湖南",
        name="关于印发《湖南省历史遗留矿山自然恢复指导意见（试行）》的通知",
        doc_number="湘自资办发[2023]58号",
    publish_date="2023-07-20", implement_date=None,
        summary="自然恢复认定标准：无地质灾害或无威胁对象、无水土污染、设备拆除、符合规划、植被覆盖度
    >75%、连续3年正向增长",
        rules=[("植被覆盖度", ">", "75", "必要条件", "%"),
               ("植被覆盖率趋势", "==", "连续3年正向增长", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("符合国土空间规划", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="新疆",
        name="自治区历史遗留废弃矿山自然恢复方式销号图斑工作指南（试行）",
        doc_number="新自然资源办函〔2024〕219号",
    publish_date="2024-12-20", implement_date=None,
        summary="荒漠区沙地/裸土地/裸岩石砾地满足无地质灾害、地形协调、无水土污染即可判定",
        rules=[("植被覆盖度", ">=", "周边30%", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("地形平缓地貌稳定", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="广西",
        name="广西壮族自治区历史遗留矿山自然恢复技术指南",
        doc_number="桂自然资办〔2024〕250号",
    publish_date="2024-12-30", implement_date=None,
        summary="除高陡边坡和裸露岩体外，水土水热条件良好、植被自然恢复趋势明显",
        rules=[("无地质灾害", "==", "是", "必要条件", None),
               ("植被自然恢复趋势", "==", "明显", "必要条件", None),
               ("生产设备拆除完毕", "==", "是", "必要条件", None)]),
```

```
    dict(category="政策文件", province="西藏",
         name="西藏自治区历史遗留废弃矿山生态自然恢复认定指导意见（试行）",
         doc_number="藏自然资发〔２０２５〕１９号",
    publish_date="２０２５-０９-２４", implement_date=None,
         summary="含修复成本高、生态修复效益不显著等适用条件",
         rules=[("责任主体", "==", "政府", "必要条件", None),
                ("地形地貌完整与周边协调", "==", "是", "必要条件", None),
                ("无地质灾害", "==", "是", "必要条件", None)]),
]
CASE = {
    "name": "玉田县典型历史遗留废弃矿山案例",
    "patch_name": "玉田县孤树镇白云岩采石场",
    "area_ha": 5.6,
    "mineral_type": "白云岩",
    "units": [
        {
            "unit_type": "岩质边坡", "ratio": 0.1386,
            "data": {"coverage": 68.88, "shannon": 0.00,
                     "slope": 63.33, "aspect": "阴阳坡"},
        },
        {
            "unit_type": "碎石边坡", "ratio": 0.6295,
            "data": {"soil_ratio": 0.34, "thickness": 10,
    "ph": 9.24,
                     "organic_matter": 4.89,
                     "heavy_metals": {"Hg": 0.05, "As":
    5.17, "Ni": 12.60,
                                      "Cu": 11.70, "Cd":
    0.04, "Pb": 6.50},
                     "coverage": 87.92, "shannon": 0.60,
                     "slope": 28.75, "aspect": "阳坡"},
        },
        {
            "unit_type": "平台", "ratio": 0.2319,
            "data": {"soil_ratio": 0.56, "thickness": 30,
    "ph": 8.72,
                     "organic_matter": 10.92,
                     "heavy_metals": {"Hg": 0.02, "As":
    3.13, "Ni": 12.33,
                                      "Cu": 11.14, "Cd":
    0.09, "Pb": 9.98},
                     "coverage": 58.42, "shannon": 0.72},
        },
    ],
}
def clean_case():
    """清空案例相关数据（幂等重跑用）。
    indicator_data / score_details 等表未设级联删除，需显式清理。
    """
    from app.models import EvaluationResult, ScoreDetail,
```

```
    ConstraintCheck                                                    4001
    ScoreDetail.query.delete()                                         4002
    EvaluationResult.query.delete()                                    4003
    ConstraintCheck.query.delete()                                     4004
    IndicatorData.query.delete()                                       4005
    MicroUnit.query.delete()                                           4006
    ReferenceEcosystem.query.delete()                                  4007
    MinePatch.query.delete()                                           4008
    Project.query.delete()                                             4009
    db.session.commit()                                                4010
def seed():                                                            4011
    with app.app_context():                                            4012
        db.create_all()          #  全新环境下先建表，保证本脚本可独立运行    4013
        clean_case()                                                   4014
        admin = User.query.filter_by(username="admin").first           4015
()                                                                     4016
        if not admin:                                                  4017
            from werkzeug.security import                              4018
generate_password_hash                                                 4019
            admin = User(username="admin",                             4020
                         password_hash=generate_password_has           4021
h("admin123"),                                                         4022
                         role="admin")                                 4023
            db.session.add(admin)                                      4024
            db.session.commit()                                        4025
        project = Project(name=CASE["name"],                           4026
code="DEMO-001",                                                       4027
                          description="论文第 5 节典型案例",             4028
created_by=admin.id)                                                   4029
        db.session.add(project)                                        4030
        db.session.flush()                                             4031
        patch = MinePatch(project_id=project.id,                       4032
name=CASE["patch_name"],                                               4033
                          admin_region="河北省唐山市玉田县",             4034
                          area_ha=CASE["area_ha"],                     4035
                          mineral_type=CASE["mineral_type"],           4036
                          mining_method="露天开采",                    4037
stop_years=None,                                                       4038
                          responsible_entity_missing=True)             4039
        db.session.add(patch)                                          4040
        db.session.flush()                                             4041
        ref = ReferenceEcosystem(patch_id=patch.id,                    4042
                                 coverage_benchmark=75.0,              4043
shannon_benchmark=0.7)                                                 4044
        db.session.add(ref)                                            4045
        for u in CASE["units"]:                                        4046
            unit = MicroUnit(patch_id=patch.id,                        4047
unit_type=u["unit_type"],                                              4048
                             ratio=u["ratio"], area=None)              4049
            db.session.add(unit)                                       4050
```

```
                db.session.flush()                                          4051
                d = u["data"]                                               4052
                ind = IndicatorData(                                        4053
                    unit_id=unit.id,                                        4054
                    soil_ratio=d.get("soil_ratio"),                         4055
                    thickness=d.get("thickness"),                           4056
                    ph=d.get("ph"),                                         4057
                    organic_matter=d.get("organic_matter"),                 4058
                    heavy_metals=json.dumps(d["heavy_metals"],              4059
    ensure_ascii=False)                                                     4060
                                        if d.get("heavy_metals") else       4061
    None,                                                                   4062
                    coverage=d.get("coverage"),                             4063
                    shannon=d.get("shannon"),                               4064
                    slope=d.get("slope"),                                   4065
                    aspect=d.get("aspect"),                                 4066
                )                                                           4067
                db.session.add(ind)                                         4068
            db.session.commit()                                             4069
            print("已写入案例：", CASE["name"])                              4070
            print("   图斑：", CASE["patch_name"])                          4071
            print("   微地质单元：岩质边坡 13.86% / 碎石边坡 62.95% / 平台    4072
    23.19%")                                                                4073
            print("   管理员账号：admin / admin123")                        4074
            seed_standards()                                                4075
def seed_standards():                                                       4076
    """写入论文表 1 的 11 项标准与政策文件及其结构化规则。"""                4077
    StandardRule.query.delete()                                             4078
    Standard.query.delete()                                                 4079
    db.session.commit()                                                     4080
    for s in STANDARDS:                                                     4081
        std = Standard(category=s["category"],                              4082
    province=s["province"],                                                 4083
                       name=s["name"],                                      4084
    doc_number=s["doc_number"],                                             4085
                       publish_date=s["publish_date"],                      4086
                       implement_date=s["implement_date"],                  4087
                       summary=s["summary"])                                4088
        db.session.add(std)                                                 4089
        db.session.flush()                                                  4090
        for item, op, thr, ctype, note in s["rules"]:                       4091
            db.session.add(StandardRule(standard_id=std.id,                 4092
    item=item,                                                              4093
                                        operator=op,                        4094
    threshold=thr,                                                          4095
                                                                            4096
    condition_type=ctype, note=note))                                       4097
    db.session.commit()                                                     4098
    print("已写入标准/政策文件：%d 项，规则 %d 条"                          4099
          % (len(STANDARDS), sum(len(s["rules"]) for s in                   4100
```

```
    STANDARDS)))                                                    4101
if __name__ == "__main__":                                          4102
    seed()                                                          4103
# ==== test_ahp.py ====                                             4104
# -*- coding: utf-8 -*-                                             4105
"""地基自检：验证 AHP 引擎与应用可启动。"""                                  4106
import sys                                                          4107
sys.stdout.reconfigure(encoding="utf-8")                            4108
import numpy as np                                                  4109
from app.services import ahp_service as ahp                         4110
LINE = "=" * 66                                                     4111
# ------------------------------------------------------------------4112
    ------------------                                              4113
# 1) 完全一致矩阵 → 权重应精确还原，CR=0                                    4114
# ------------------------------------------------------------------4115
    ------------------                                              4116
print(LINE)                                                         4117
print("测试 1  完全一致判断矩阵的权重还原与一致性")                              4118
print(LINE)                                                         4119
# 由权重构造完全一致矩阵：a_ij = w_i / w_j                                 4120
w_true = np.array([0.44, 0.31, 0.25])                               4121
A = np.outer(w_true, 1.0 / w_true)                                  4122
r = ahp.weights_and_consistency(A)                                  4123
print("  判断矩阵 A：")                                                 4124
for row in A:                                                       4125
    print("    " + "  ".join(f"{v:8.4f}" for v in row))              4126
print()                                                             4127
print("  计算权重 :", "  ".join(f"{v:.4f}" for v in                  4128
    r["weights"]))                                                  4129
print("  真实权重 :", "  ".join(f"{v:.4f}" for v in w_true))          4130
print("  λmax = %.6f   CI = %.3e   CR = %.3e   一致: %s"              4131
      % (r["lambda_max"], r["ci"], r["cr"],                         4132
    r["consistent"]))                                               4133
assert np.allclose(r["weights"], w_true, atol=1e-6),                4134
    "权重未精确还原"                                                    4135
assert r["consistent"], "完全一致矩阵应通过检验"                           4136
# ------------------------------------------------------------------4137
    ------------------                                              4138
# 2) 上三角输入 → 完整矩阵 → 一致性定位                                    4139
# ------------------------------------------------------------------4140
    ------------------                                              4141
print()                                                             4142
print(LINE)                                                         4143
print("测试 2  上三角录入与最不一致项定位")                                  4144
print(LINE)                                                         4145
# 一个不太一致的 3 阶矩阵：上三角 [2, 5, 3]（a12=2, a13=5, a23=3）          4146
M = ahp.matrix_from_upper([2, 5, 3])                                4147
r2 = ahp.weights_and_consistency(M)                                 4148
bad = ahp.find_most_inconsistent(M)                                 4149
print("  由 [a12=2, a13=5, a23=3] 构造矩阵：")                          4150
```

```
for row in M:
    print("    " + "  ".join(f"{v:8.4f}" for v in row))
print("  权重:", "  ".join(f"{v:.4f}" for v in r2["weights"]),
      "  CR = %.3f  一致: %s" % (r2["cr"], r2["consistent"]))
print("  最不一致比较项: 第(%d,%d) 项,当前 %.3f,理想 %.3f"
      % (bad["i"] + 1, bad["j"] + 1, bad["value"],
    bad["expected"]))
# ------------------------------------------------------------------
    ----------------
# 3) 多专家聚合(几何平均)
# ------------------------------------------------------------------
    ----------------
print()
print(LINE)
print("测试 3  多专家判断矩阵聚合(几何平均)")
print(LINE)
m1 = ahp.matrix_from_upper([2, 4, 3])
m2 = ahp.matrix_from_upper([3, 5, 2])
agg = ahp.aggregate_matrices([m1, m2])
print("  专家1 上三角 [2,4,3],专家2 上三角 [3,5,2]")
print("  聚合后上三角: a12=%.4f a13=%.4f a23=%.4f"
      % (agg[0, 1], agg[0, 2], agg[1, 2]))
assert abs(agg[0, 1] - np.sqrt(2 * 3)) < 1e-9, "几何平均错误"
# ------------------------------------------------------------------
    ----------------
# 4) 论文表 3 权重可还原性
# ------------------------------------------------------------------
    ----------------
print()
print(LINE)
print("测试 4  论文表 3 碎石边坡一级权重 (0.44, 0.31, 0.25)")
print(LINE)
j = ahp.judge_from_weights([0.44, 0.31, 0.25])
print("  λmax = %.6f  CR = %.3e  一致: %s" %
    (j["lambda_max"], j["cr"], j["consistent"]))
assert j["consistent"], "论文权重应能作为完全一致矩阵主特征向量"
# ------------------------------------------------------------------
    ----------------
# 5) 应用可启动 + 建表
# ------------------------------------------------------------------
    ----------------
print()
print(LINE)
print("测试 5  Flask 应用启动与建表")
print(LINE)
from app import create_app
from app.models import db
app = create_app()
with app.app_context():
    db.create_all()
```

```
    tables = sorted(db.metadata.tables.keys())
print("  已创建 %d 张表：" % len(tables))
for t in tables:
    print("    -", t)
print()
print(LINE)
print("全部测试通过 √")
print(LINE)
# ==== test_evaluate.py ====
# -*- coding: utf-8 -*-
"""评价引擎自检：用论文玉田县案例跑通完整计算链。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
from app.services import evaluation_service as ev
from app.services import standardization as std
LINE = "=" * 70
# ------------------------------------------------------------------
    ------------------
# 论文第 5 节案例数据（表 5，逐字抄录）
# ------------------------------------------------------------------
    ------------------
# 三类单元面积占比：岩质边坡 １３．８６%、碎石边坡 ６２．９５%、平台 ２３．１９%
# 注：岩质边坡无土壤指标、平台无地形指标，对应"不适用"。
case = [
    {
        "unit_type": "岩质边坡",
        "ratio": 0.1386,
        "data": {
            "coverage": 68.88, "shannon": 0.00,
            "slope": 63.33, "aspect": "阴阳坡",
        },
    },
    {
        "unit_type": "碎石边坡",
        "ratio": 0.6295,
        "data": {
            "soil_ratio": 0.34, "thickness": 10, "ph":
    9.24, "organic_matter": 4.89,
            "heavy_metals": {"Hg": 0.05, "As": 5.17, "Ni":
    12.60,
                             "Cu": 11.70, "Cd": 0.04, "Pb":
    6.50},
            "coverage": 87.92, "shannon": 0.60,
            "slope": 28.75, "aspect": "阳坡",
        },
    },
    {
        "unit_type": "平台",
        "ratio": 0.2319,
        "data": {
```

```
                "soil_ratio": 0.56, "thickness": 30, "ph":
    8.72, "organic_matter": 10.92,
                "heavy_metals": {"Hg": 0.02, "As": 3.13, "Ni":
    12.33,
                                 "Cu": 11.14, "Cd": 0.09, "Pb":
    9.98},
                "coverage": 58.42, "shannon": 0.72,
            },
    },
]
# ----------------------------------------------------------------
    ----------------
# 逐单元输出标准化与得分
# ----------------------------------------------------------------
    ----------------
print(LINE)
print("玉田县案例 · 评价引擎完整计算")
print(LINE)
for u in case:
    r = ev.score_unit(u["unit_type"], u["data"])
    print()
    print("  【%s】  (面积占比 %.2f%%)" % (u["unit_type"],
    u["ratio"] * 100))
    print("  " + "-" * 60)
    for l1, score in r["l1_scores"].items():
        print("    %s 得分 = %.4f" % (l1, score))
    print("    单元评价因子 F' = %.4f" % r["unit_score"])
print()
print(LINE)
print("面积加权合成（式(9)）")
print(LINE)
res = ev.score_mine(case)
total = res["final_score"]
print()
for ur in res["unit_results"]:
    print("  %s  %.2f%% × %.4f = %.4f"
          % (ur["unit_type"], ur["ratio"] * 100,
    ur["unit_score"], ur["weighted"]))
print("  " + "-" * 60)
print("  最终评价因子 F = %.4f" % total)
print()
print("  适宜性等级：%s" % res["grade"])
print("  判定依据  ：%s" % res["grade_info"]["meaning"])
print("  推荐措施  ：%s" % res["grade_info"]["measure"])
print()
print(LINE)
print("结论")
print(LINE)
print("  - 评价引擎按论文式(8)(9) 与表 3 权重完整跑通")
print("  - 坡度标准化采用论文 25° 分界的线性折减默认值（见 standardization.py）")
```

```
print("   - 最终 F ≈ %.2f，判定为「%s」，与论文「最终适宜自然恢复」的结论一致"
      % (total, res["grade"]))
print("   - 论文表 5 所载最终得分为 0.76；差异集中在坡度、坡向的标准化口径，")
print("     该口径在 standardization.std_slope / aspect_score
    中集中定义、可配置")
print(LINE)
# ==== test_web_ahp.py ====
# -*- coding: utf-8 -*-
"""M6 AHP 网页界面自检：走通选择→录入→计算 全流程。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
from app import create_app
LINE = "=" * 66
# 测试配置：关闭 CSRF 以简化测试客户端（真实运行仍开启 CSRF）
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///:memory:"
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 建立表并创建测试用户，登录后再访问业务页面
with app.app_context():
    from app.models import db, User
    from werkzeug.security import generate_password_hash
    db.create_all()
    if not User.query.filter_by(username="admin").first():
        db.session.add(User(username="admin",
                            password_hash=generate_password_
    hash("admin123"),
                            role="admin"))
        db.session.commit()
client.post("/login", data={"username": "admin",
    "password": "admin123"})
print(LINE)
print("测试 1  首页与 AHP 入口")
print(LINE)
r = client.get("/")
print("  首页状态码：", r.status_code, "（应 200）")
assert r.status_code == 200
r = client.get("/ahp/")
print("  AHP 入口状态码：", r.status_code, "（应 200）")
assert r.status_code == 200
assert "微地质单元类型" in r.get_data(as_text=True)
print()
print(LINE)
print("测试 2  选择碎石边坡一级指标 → 跳转矩阵页")
print(LINE)
r = client.post("/ahp/", data={
```

```
        "unit_type": "碎石边坡", "level": "1", "submit": "生成判断矩阵",
    }, follow_redirects=True)
    body = r.get_data(as_text=True)
    print("  跳转后页面含「判断矩阵录入」:", "判断矩阵录入" in body)
    print("  含三个一级指标（土壤/植被/地形）:", all(n in body for n in
        ("土壤指标", "植被指标", "地形指标")))
    assert "判断矩阵录入" in body
    assert all(n in body for n in ("土壤指标", "植被指标", "地形指标"))
    print()
    print(LINE)
    print("测试 3  提交论文表 3 一致矩阵 → 还原权重")
    print(LINE)
    # 碎石边坡一级权重 (0.44, 0.31, 0.25) 对应的完全一致矩阵上三角：
    # a12 = 0.44/0.31 = 1.4194, a13 = 0.44/0.25 = 1.76, a23 =
        0.31/0.25 = 1.24
    r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
        "m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24",
    }, follow_redirects=True)
    body = r.get_data(as_text=True)
    print("  结果页含「计算结果」:", "计算结果" in body)
    print("  含权重 0.44:", "0.44" in body, " 含 0.31:", "0.31" in
        body, " 含 0.25:", "0.25" in body)
    print("  含一致性检验通过:", "通过" in body)
    assert "计算结果" in body
    assert "0.44" in body and "0.31" in body and "0.25" in body
    print()
    print(LINE)
    print("测试 4  不一致矩阵应提示不通过并定位")
    print(LINE)
    # 明显不一致：a12=9, a13=1, a23=9
    r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
        "m_0_1": "9", "m_0_2": "1", "m_1_2": "9",
    }, follow_redirects=True)
    body = r.get_data(as_text=True)
    print("  含「不通过」:", "不通过" in body)
    print("  含「建议优先调整」:", "建议优先调整" in body)
    assert "不通过" in body
    assert "建议优先调整" in body
    print()
    print(LINE)
    print("M6 网页界面全部测试通过 √")
    print(LINE)
# ==== test_web_modules.py ====
# -*- coding: utf-8 -*-
"""M1–M5 模块自检：登录、项目档案、微地质单元、标准库、限制性判定。"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
from app import create_app
LINE = "=" * 66
```

```
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        os.path.dirname(__file__), "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
print(LINE)
print("测试 1  未登录应被重定向到登录页")
print(LINE)
r = client.get("/project/")
print("  /project/ 状态码:", r.status_code, " 重定向到:",
    r.headers.get("Location"))
assert r.status_code == 302 and "login" in
    r.headers.get("Location", "")
print()
print(LINE)
print("测试 2  登录")
print(LINE)
r = client.post("/login", data={"username": "admin",
    "password": "admin123"},
                follow_redirects=True)
body = r.get_data(as_text=True)
print("  登录后含「系统功能」:", "系统功能" in body)
print("  含用户名 admin:", "admin" in body)
assert "系统功能" in body
print()
print(LINE)
print("测试 3  M2 项目与矿山档案")
print(LINE)
r = client.get("/project/")
body = r.get_data(as_text=True)
print("  项目列表状态码:", r.status_code)
print("  含案例项目:", "玉田县典型历史遗留废弃矿山案例" in body)
assert "玉田县典型历史遗留废弃矿山案例" in body
# 新建项目
client.post("/project/add", data={"name": "测试项目A", "code":
    "T-001"})
r = client.get("/project/")
print("  新建项目后含「测试项目A」:", "测试项目A" in r.get_data(as_text=True
    ))
assert "测试项目A" in r.get_data(as_text=True)
# 清理测试数据，避免污染演示库
with app.app_context():
    from app.models import db, Project
    for p in Project.query.filter_by(name="测试项目A").all():
        db.session.delete(p)
    db.session.commit()
```

```
print("    已清理测试项目")
print()
print(LINE)
print("测试 4   M5 微地质单元（面积占比自动计算）")
print(LINE)
from app.models import MinePatch
with app.app_context():
    patch = MinePatch.query.filter_by(name="玉田县孤树镇白云岩采石场").first()
    pid = patch.id
r = client.get("/unit/%d" % pid)
body = r.get_data(as_text=True)
print("    单元列表含三类单元：", all(t in body for t in ("岩质边坡",
    "碎石边坡", "平台")))
print("    含占比合计：", "占比合计" in body)
assert "岩质边坡" in body and "占比合计" in body
print()
print(LINE)
print("测试 5   M3 标准知识库与省际口径比对")
print(LINE)
r = client.get("/standard/")
body = r.get_data(as_text=True)
print("    含辽宁/湖南/新疆标准：", all(p in body for p in ("辽宁", "湖南",
    "新疆")))
assert "辽宁" in body
r = client.get("/standard/compare")
body = r.get_data(as_text=True)
print("    口径比对页含「植被覆盖度」分组：", "植被覆盖度" in body)
print("    含多省份并列：", "湖南" in body and "新疆" in body)
assert "植被覆盖度" in body
print()
print(LINE)
print("测试 6   M4 限制性指标判定（一票否决）")
print(LINE)
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("    含五项条件：", all(x in body for x in ("政策约束", "区位约束",
    "土壤污染", "地质灾害", "耕地属性")))
assert all(x in body for x in ("政策约束", "区位约束", "土壤污染",
    "地质灾害", "耕地属性"))
# 全部通过
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "1",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("    全通过后显示「通过，可进入定量评价」：", "可进入定量评价" in body)
assert "可进入定量评价" in body
# 一项不通过
```

```
client.post("/constraint/%d" % pid, data={                              4501
    "policy_pass": "1", "location_pass": "0",                           4502
    "pollution_pass": "1",                                              4503
    "geo_pass": "1", "farmland_pass": "1"})                             4504
r = client.get("/constraint/%d" % pid)                                  4505
body = r.get_data(as_text=True)                                         4506
print("  一项不通过后显示「不通过」:", "不通过" in body)                    4507
print("  含失败项「区位约束」:", "区位约束" in body)                        4508
assert "不通过" in body                                                  4509
print()                                                                 4510
print(LINE)                                                             4511
print("M1–M5 全部测试通过 √")                                           4512
print(LINE)                                                             4513
# ==== test_web_evaluate.py ====                                        4514
# -*- coding: utf-8 -*-                                                 4515
"""M7 + M8 端到端自检：录数据 → 执行评价 → 查看结果。"""                  4516
import os                                                               4517
import sys                                                              4518
sys.stdout.reconfigure(encoding="utf-8")                                4519
from app import create_app                                              4520
LINE = "=" * 66                                                         4521
# 与真实配置共用同一个 app.db，但关闭 CSRF 便于测试客户端                 4522
class TestConfig:                                                       4523
    SECRET_KEY = "test"                                                 4524
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(              4525
        os.path.dirname(__file__), "app", "app.db")                     4526
    SQLALCHEMY_TRACK_MODIFICATIONS = False                              4527
    WTF_CSRF_ENABLED = False                                            4528
    TESTING = True                                                      4529
app = create_app(TestConfig)                                            4530
client = app.test_client()                                              4531
# 登录（案例种子已创建 admin 用户）                                      4532
client.post("/login", data={"username": "admin",                        4533
    "password": "admin123"})                                            4534
print(LINE)                                                             4535
print("测试 1  M7 指标数据列表")                                        4536
print(LINE)                                                             4537
r = client.get("/indicator/")                                           4538
body = r.get_data(as_text=True)                                         4539
print("  状态码:", r.status_code)                                       4540
print("  含案例图斑:", "玉田县孤树镇白云岩采石场" in body)               4541
print("  含三个单元:", all(n in body for n in ("岩质边坡", "碎石边坡",   4542
    "平台")))                                                           4543
assert r.status_code == 200                                             4544
assert "岩质边坡" in body and "碎石边坡" in body and "平台" in body      4545
print()                                                                 4546
print(LINE)                                                             4547
print("测试 2  M7 指标数据录入页（按单元类型显示字段）")                4548
print(LINE)                                                             4549
from app.models import MicroUnit, MinePatch                             4550
```

```
with app.app_context():
    unit_rock = MicroUnit.query.filter_by(unit_type="岩质边坡").first()
    unit_gravel = MicroUnit.query.filter_by(unit_type="碎石边坡").first()
r = client.get("/indicator/%d" % unit_rock.id)
b_rock = r.get_data(as_text=True)
print("  岩质边坡页：无「土石比」字段：", "土石比" not in b_rock, "（应 True）")
print("  岩质边坡页：含「植被覆盖度」：", "植被覆盖度" in b_rock)
assert "土石比" not in b_rock and "植被覆盖度" in b_rock
r = client.get("/indicator/%d" % unit_gravel.id)
b_gravel = r.get_data(as_text=True)
print("  碎石边坡页：含「土石比」字段：", "土石比" in b_gravel)
print("  碎石边坡页：含 8 种重金属：", all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn")))
assert "土石比" in b_gravel and all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn"))
print()
print(LINE)
print("测试 3  M8 执行评价")
print(LINE)
with app.app_context():
    patch = MinePatch.query.first()
r = client.post("/evaluate/%d" % patch.id, data={},
    follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转至结果页：", "适宜性评价结果" in body)
print("  含最终评价因子 F：", "最终评价因子 F" in body)
print("  含等级「适宜」：", "适宜" in body)
print("  含推荐措施「自然恢复」：", "自然恢复" in body)
assert "最终评价因子 F" in body and "自然恢复" in body
print()
print(LINE)
print("测试 4  结果持久化")
print(LINE)
from app.models import EvaluationResult
with app.app_context():
    res = EvaluationResult.query.order_by(EvaluationResult.id.desc()).first()
    print("  最终得分 F =", res.final_score)
    print("  等级 =", res.grade)
    assert res.grade == "适宜"
    assert 0.7 <= res.final_score < 1.0
print()
print(LINE)
print("M7 + M8 端到端全部测试通过 √")
print(LINE)
# ==== test_web_report.py ====
# -*- coding: utf-8 -*-
"""M9 报告输出自检：得分明细页 + Word 导出。"""
```

```
import os
import sys
import io
sys.stdout.reconfigure(encoding="utf-8")
from app import create_app
LINE = "=" * 66
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        os.path.dirname(__file__), "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 登录（案例种子已创建 admin 用户）
client.post("/login", data={"username": "admin",
    "password": "admin123"})
from app.models import MinePatch
with app.app_context():
    patch = MinePatch.query.first()
    pid = patch.id
print(LINE)
print("测试 1  报告列表")
print(LINE)
r = client.get("/report/")
print("  状态码:", r.status_code)
assert r.status_code == 200
print()
print(LINE)
print("测试 2  得分明细页")
print(LINE)
r = client.get("/report/%d" % pid)
body = r.get_data(as_text=True)
print("  状态码:", r.status_code)
print("  含「得分明细」:", "得分明细" in body)
print("  含「最终评价因子 F」:", "最终评价因子 F" in body)
print("  含二级指标明细（植被覆盖度/坡度/坡向）:", all(n in body for n in
    ("植被覆盖度", "坡度", "坡向")))
print("  含权重/标准化值列:", "标准化值" in body and "贡献" in body)
assert r.status_code == 200
assert "植被覆盖度" in body and "标准化值" in body
print()
print(LINE)
print("测试 3  Word 报告导出")
print(LINE)
r = client.get("/report/%d/word" % pid)
print("  状态码:", r.status_code)
print("  Content-Type:", r.headers.get("Content-Type",
    "")[:60])
```

```
data = r.data                                                                4651
print("   文件大小：", len(data), "字节")                                      4652
print("   是否为有效 docx（PK 头）：", data[:2] == b"PK")                      4653
assert r.status_code == 200 and data[:2] == b"PK"                            4654
# 用 python-docx 打开验证内容                                                4655
import docx                                                                  4656
d = docx.Document(io.BytesIO(data))                                          4657
text = "\n".join(p.text for p in d.paragraphs)                               4658
print("   含报告标题：", "自然恢复适宜性评价报告" in text)                     4659
print("   含评价结论章节：", "评价结论" in text)                               4660
print("   含得分明细章节：", "得分明细" in text)                               4661
print("   段落数：", len(d.paragraphs), "  表格数：", len(d.tables))          4662
assert "自然恢复适宜性评价报告" in text and len(d.tables) >= 3                4663
print()                                                                      4664
print(LINE)                                                                  4665
print("M9 报告输出全部测试通过 √")                                          4666
print(LINE)                                                                  4667
# ==== test_ui_fixes.py ====                                                 4668
# -*- coding: utf-8 -*-                                                      4669
"""界面修复自检：「记住我」与各模块删除功能。"""                             4670
import os                                                                    4671
import re                                                                    4672
import sys                                                                   4673
from app import create_app                                                   4674
sys.stdout.reconfigure(encoding="utf-8")                                     4675
LINE = "=" * 70                                                              4676
class TestConfig:                                                            4677
    SECRET_KEY = "test"                                                      4678
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(                   4679
        os.path.dirname(__file__), "app", "app.db")                          4680
    SQLALCHEMY_TRACK_MODIFICATIONS = False                                   4681
    WTF_CSRF_ENABLED = False                                                 4682
    TESTING = True                                                           4683
    REMEMBER_COOKIE_DURATION = __import__("datetime").timede                 4684
    lta(days=30)                                                             4685
app = create_app(TestConfig)                                                 4686
client = app.test_client()                                                   4687
# ------------------------------------------------------------------------   4688
    ------------------                                                       4689
print(LINE)                                                                  4690
print("一、「记住我」")                                                      4691
print(LINE)                                                                  4692
html = client.get("/login").get_data(as_text=True)                           4693
# 1） 复选框必须有可点击关联的 label（这是原先失效的根因）                   4694
wrapped = re.search(r'<label[^>]*class="checkbox"[^>]*>\s*<i                 4695
    nput[^>]*name="remember"',                                               4696
                        html) is not None                                    4697
print("   %s 「记住我」文字与复选框包在同一 label 内" % ("[符合]" if wrapped  4698
    else "[不符合]"))                                                        4699
assert wrapped, "复选框未与文字关联，点击文字不会勾选"                       4700
```

```
has_for = 'for="remember"' in html
print("  %s label 带 for 属性指向复选框" % ("[符合]" if has_for else
    "[不符合]"))
# 2) 勾选后应下发带过期时间的 remember_token
c = app.test_client()
r = c.post("/login", data={"username": "admin", "password":
    "admin123",
                           "remember": "y"},
    follow_redirects=False)
cookies = r.headers.getlist("Set-Cookie")
rt = [h for h in cookies if h.startswith("remember_token=")]
print("  %s 勾选后下发 remember_token" % ("[符合]" if rt else
    "[不符合]"))
assert rt, "未下发 remember_token"
exp = re.search(r"Expires=([^;]+)", rt[0])
print("  %s 该 cookie 带过期时间：%s"
      % ("[符合]" if exp else "[不符合]", exp.group(1) if exp
    else "无（会话 cookie）"))
assert exp, "remember_token 无过期时间，关浏览器即失效"
# 3) 不勾选时不应下发 remember_token
c2 = app.test_client()
r2 = c2.post("/login", data={"username": "admin",
    "password": "admin123"},
                follow_redirects=False)
rt2 = [h for h in r2.headers.getlist("Set-Cookie") if
    h.startswith("remember_token=")]
print("  %s 未勾选时不下发 remember_token" % ("[符合]" if not rt2
    else "[不符合]"))
assert not rt2
# 4) 携带 remember_token 的新会话应自动登录
c3 = app.test_client()
c3.set_cookie("remember_token",
              rt[0].split(";")[0].split("=", 1)[1],
    domain="localhost")
r3 = c3.get("/", follow_redirects=False)
print("  %s 仅凭 remember_token 访问首页可自动登录（状态码 %d）"
      % ("[符合]" if r3.status_code == 200 else "[不符合]",
    r3.status_code))
assert r3.status_code == 200
# ------------------------------------------------------------------
    ----------------
print()
print(LINE)
print("二、删除功能")
print(LINE)
client.post("/login", data={"username": "admin",
    "password": "admin123"})
# 项目：列表页须有删除按钮
html = client.get("/project/").get_data(as_text=True)
has_del = "/delete" in html and "btn--danger" in html
```

```
print("  %s 项目列表含删除按钮" % ("[符合]" if has_del else "[不符合]"))
assert has_del, "项目列表缺少删除按钮"
print("  %s 删除前有二次确认" % ("[符合]" if "confirm(" in html else
    "[不符合]"))
assert "confirm(" in html
# 图斑列表
with app.app_context():
    from app.models import MinePatch, Project
    patch = MinePatch.query.first()
    patch_id = patch.id
html = client.get("/project/%d/patches" %
    patch_id).get_data(as_text=True)
print("  %s 图斑列表含删除按钮" % ("[符合]" if "/project/patch/" in
    html and "btn--danger" in html else "[不符合]"))
assert "btn--danger" in html
# 微地质单元
html = client.get("/unit/%d" % patch_id).get_data(as_text=Tr
    ue)
print("  %s 微单元列表含删除按钮" % ("[符合]" if "/unit/delete/" in
    html else "[不符合]"))
assert "/unit/delete/" in html
print("  %s 微单元编辑为独立页面（表单不再嵌在表格行内）"
      % ("[符合]" if "/unit/edit/" in html else "[不符合]"))
# 标准
html = client.get("/standard/").get_data(as_text=True)
print("  %s 标准列表含删除按钮" % ("[符合]" if "btn--danger" in html
    else "[不符合]"))
assert "btn--danger" in html
# 实际执行一次删除：新建项目 → 删除 → 确认消失
client.post("/project/add", data={"name": "待删除测试项目",
    "code": "DEL-1"})
with app.app_context():
    from app.models import db, Project
    tmp = Project.query.filter_by(name="待删除测试项目").first()
    tmp_id = tmp.id
html = client.get("/project/").get_data(as_text=True)
print("  %s 新建项目成功" % ("[符合]" if "待删除测试项目" in html else
    "[不符合]"))
r = client.post("/project/%d/delete" % tmp_id,
    follow_redirects=True)
html = r.get_data(as_text=True)
gone = "待删除测试项目" not in html
print("  %s 删除项目后列表中不再出现" % ("[符合]" if gone else "[不符合]"))
assert gone
print("  %s 有删除成功提示" % ("[符合]" if "已删除" in html else
    "[不符合]"))
print()
print(LINE)
print("界面修复自检全部通过 √")
print(LINE)
```